

递归循环 Transformer

Recurrent Looped Transformer

Yifan Zhang Jichen Feng Shihan Qin

yifanzhangresearch@gmail.com

2026 年 9 月 12 日*

摘要

递归循环 Transformer (RLT-1) 通过门控融合，将上一个 token 的最终解码器状态反馈到当前 token 的解码器输入。因果编码器提供 token 表示和全局键值记忆，解码器则维护局部滑动窗口注意力 (SWA) 缓存。这一递归使计算路径随序列长度增长，同时保持每个 token 的模块执行次数不变，但训练和提示预填充均需按顺序更新解码器。我们在六个算法任务上比较五种八层编码器与解码器层数分配和仅含解码器的 Transformer，使用三个初始化种子以及共享的训练与测试数据。部分 RLT-1 配置更早学会奇偶校验，其中 $5+3$ 和 $7+1$ 在全部三个种子上均保持 256 位时 100% 的准确率。在长度为 512 的交换置换跟踪中， $4+4$ 的最终状态准确率为 $55.70 \pm 25.78\%$ ，Transformer 为 $0.85 \pm 0.30\%$ (均值 $\pm$ 样本标准差)。收益取决于任务和层数分配：教师强制加法准确率在超出训练位数后下降，无括号模运算对初始化敏感，标准置换跟踪仍然困难。

项目主页: <https://github.com/yifanzhang-pro/recurrent-looped-transformer>

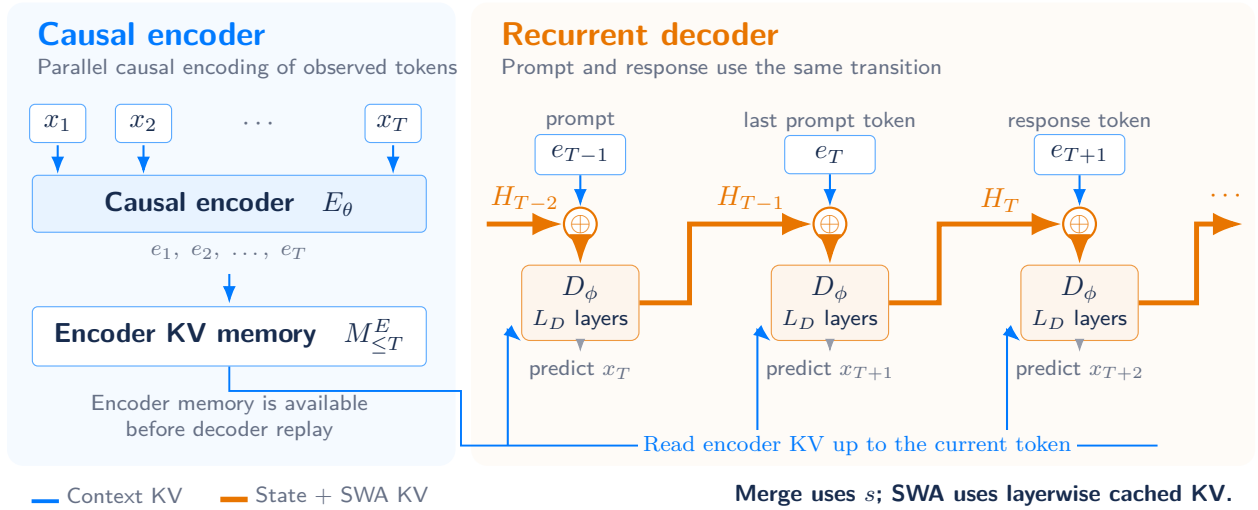


图 1 跨越提示与回复的递归计算。编码器构建因果键值 (KV) 记忆，解码器按 token 顺序处理。橙色箭头传递解码器状态 $H_t = (s_t, C_t^D)$: 融合操作读取 s_t ，每个 SWA 层读取各自的缓存。蓝色箭头提供截至当前位置的编码器记忆。图中展示提示的最后两次更新和回复的第一次更新。

*修订于 2026 年 9 月 17 日；中文译本对应同日英文公开版。

1 引言

算法状态跟踪要求模型在每个输入到来时更新状态。递归网络直接表达了这一过程：一次更新的输出成为下一次更新的输入。在 Transformer 语言模型中加入这样的连接，可让后续 token 沿随序列长度增长的路径，继续利用先前 token 已完成的计算。由此产生两个具体问题：反馈应放在何处，每次更新应分配多少计算？

本文研究递归循环 Transformer (RLT-1)，将反馈放在因果编码器与 Transformer 解码器之间。编码器构建 token 表示和全局键值记忆。在每个位置，门控融合将当前编码器表示与前一步解码器的最终输出结合。随后，解码器关注编码器前缀和自身近期激活，并预测下一个 token。提示与回复使用同一状态转移（图 1）。

编码器与解码器承担不同的计算任务。已知 token 可以并行编码，而解码器必须按 token 顺序更新。解码器深度为 L_D 时，处理 t 个 token 会形成穿过 tL_D 个解码器模块的递归路径。在总层数不变时，将层从编码器移至解码器会延长这条路径，也会增加训练与预填充的顺序计算量。因此，我们比较多种层数分配，而不固定采用等深的编码器与解码器。

六个算法任务、三个初始化种子的实验表明，较优的层数分配取决于所学习的计算。 $5 + 3$ 和 $7 + 1$ 在每个种子上都能将奇偶校验泛化至 256 位，而分配更多解码器层的模型更擅长较长的交换置换序列。这些表现并未普遍延伸到其他任务：加法在超出训练位数后退化，无括号模运算在不同初始化之间差异较大。实验比较五种层数分配与仅含解码器的 Transformer，记录各任务在训练长度范围内的表现。

RLT-1 将编码器记忆复用 (Sun et al., 2024) 与跨 token 反馈相结合；Feedback Transformer、Recurrent Transformer、Full-bandwidth Transformer、T²MLR 和 Latent Recurrent Transformer 已探索了这类反馈机制 (Fan et al., 2020; Oncescu et al., 2026; Wang et al., 2026; Cai et al., 2026; Huang et al., 2026)。本文研究贯穿整个解码器的反馈，以及编码与递归更新之间的深度分配。附录给出训练、缓存复用和策略重放的完整规范，并描述尚未经实验检验的扩展 RLT-2：它在一个 chunk 内共享反馈状态，以并行解码已知 token。

2 方法

本节定义实验所用的 RLT-1。附录 I 给出 RLT-2 的分块扩展。

2.1 序列与状态

设 $x_{1:S}$ 为以 $x_1 = \text{BOS}$ 开始的一条独立序列。推理时， $x_{1:T}$ 为提示，后续 token 构成回复。模型在 T 两侧使用同一条件分布。令 L_E 和 L_D 分别为编码器与解码器深度， d 为残差宽度。编码器表示 $e_t \in \mathbb{R}^d$ 和递归输出 $s_t \in \mathbb{R}^d$ 均写作列向量。

完整的解码器状态为 $H_t = (s_t, C_t^D)$ 。缓存 C_t^D 保存各解码器 SWA 层保留的键值投影。窗口大小 $W \geq 1$ 包含当前 token，因此每层在一次更新后至多保留 $W - 1$ 个历史位置。所有参数（包括可学习初始状态 s_* ）统记为 Θ 。编码器和解码器分别记作 E_θ 和 D_ϕ 。

2.2 因果编码器与记忆

对于已观察到的前缀，计算

$$e_{1:T} = E_\theta(x_{1:T}). \quad (2.1)$$

每个编码器层内的位置可使用因果掩码一起处理，而编码器各层仍按深度顺序执行。对于解码器记忆组 $g \in \{1, \dots, G\}$ ，构造

$$k_t^g = \mathcal{P}_K^g(e_t, t), \quad v_t^g = W_V^g \text{RMSNorm}_E(e_t), \quad M_{\leq t}^g = \{(k_j^g, v_j^g)\}_{j=1}^t. \quad (2.2)$$

键映射包含归一化、投影以及所采用的位置变换。解码器第 ℓ 层读取组 $g(\ell)$ ： $G = 1$ 表示各层共享记忆， $G = L_D$ 则允许各层使用独立投影。解码器通过不同的注意力子层读取两类存储。交叉注意力读取覆盖整个前缀的编码器记忆 $M_{\leq t}$ ；因果 SWA 读取有界窗口内的解码器 KV。

2.3 每个 token 使用同一状态转移

在 BOS 之前仅初始化一次：

$$H_0 = (s_\star, \emptyset). \quad (2.3)$$

对每个已观察到或已采样的 token，执行

$$u_t = \text{Merge}(e_t, s_{t-1}), \quad (2.4)$$

$$H_t = (s_t, C_t^D) = D_\phi(u_t; M_{\leq t}, C_{t-1}^D, t), \quad t \geq 1, \quad (2.5)$$

$$p_\Theta(x_{t+1} \mid x_{1:t}) = \text{softmax}(W_o \text{RMSNorm}_o(s_t))_{x_{t+1}}. \quad (2.6)$$

对 $H = (s, C^D)$ ，定义 $F_t(H) = D_\phi(\text{Merge}(e_t, s); M_{\leq t}, C^D, t)$ 。编码器特征可以在解码器运行前计算。随后，为预测第一个回复 token，解码器必须处理提示中的每个位置：

$$H_T = F_T \circ F_{T-1} \circ \dots \circ F_1(H_0). \quad (2.7)$$

生成从 H_T 继续。根据 s_T 采样 x_{T+1} 后，增量编码该 token，追加由编码器生成的 KV，并计算 $H_{T+1} = F_{T+1}(H_T)$ ，其中包含各解码器 SWA 缓存的更新。在提示与回复的服务边界处， H_T 的两个分量都不重置。

2.4 提示预填充与增量解码

预填充先计算提示的因果编码器表示与记忆，再依次求出 $H_1, \dots, H_T$ 。即使整个提示的记忆已经可用，解码器在位置 t 的注意力仍限制为 $M_{\leq t}$ 。生成阶段使用

$$(e_t, C_t^E) = E_\theta^{\text{step}}(x_t, C_{t-1}^E) \quad (2.8)$$

更新编码器缓存 C^E ，随后追加记忆并更新解码器。若保留编码器缓存，已观察到的 token 可以分块编码。解码器仍逐 token 顺序处理：每次更新都产生后续位置需要的隐藏状态与 SWA KV。

2.5 融合与解码器模块

采用以下门控融合：

$$r_{t-1} = \text{RMSNorm}_s(s_{t-1}), \quad (2.9)$$

$$g_t = \sigma(W_g[e_t; r_{t-1}] + b_g), \quad (2.10)$$

$$u_t = e_t + \alpha g_t \odot W_s r_{t-1}. \quad (2.11)$$

其中 $W_g \in \mathbb{R}^{d \times 2d}$ ， $W_s \in \mathbb{R}^{d \times d}$ ， α 控制反馈幅度。实验使用 $\alpha = 0.1$ 。标量门控或低秩 W_s 可以降低融合成本。

从 $z_t^0 = u_t$ 开始，每个解码器模块依次应用因果 SWA、编码器记忆交叉注意力和前馈网络 (FFN)：

$$q_t^{D,\ell} = \mathcal{P}_Q^{D,\ell}(z_t^{\ell-1}, t), \quad (2.12)$$

$$k_t^{D,\ell} = \mathcal{P}_K^{D,\ell}(z_t^{\ell-1}, t), \quad v_t^{D,\ell} = W_V^{D,\ell} \text{RMSNorm}_{S,\ell}(z_t^{\ell-1}), \quad (2.13)$$

$$b_t^\ell = z_t^{\ell-1} + \text{Attn}_\ell^D \left(q_t^{D,\ell}, \{(k_j^{D,\ell}, v_j^{D,\ell})\}_{j=\max(1,t-W+1)}^t \right), \quad (2.14)$$

$$a_t^\ell = b_t^\ell + \text{Attn}_\ell^M \left(\mathcal{P}_Q^{M,\ell}(b_t^\ell, t), M_{\leq t}^{g(\ell)} \right), \quad (2.15)$$

$$z_t^\ell = a_t^\ell + \text{FFN}_\ell(\text{RMSNorm}_{D,\ell}(a_t^\ell)), \quad s_t = z_t^{L_D}. \quad (2.16)$$

注意力算子包含输出投影；查询与键映射包含各自的归一化和位置变换。各层在执行 SWA 前，先由该层输入形成当前位置的 KV，因此关注当前位置不会引入循环依赖。历史解码器 KV 来自 C_{t-1}^D 。更新后，在 C_t^D 中保留位置 $\max(1, t - W + 2), \dots, t$ ；当 $W = 1$ 时，该集合为空。预填充、采样和重放采用相同的位置元数据约定。不同的子层顺序定义不同的变体，必须在所有执行模式中保持一致。

图 6 展开了单次 token 更新中的编码器与解码器堆栈。

2.6 递归深度与执行成本

处理 t 个 token 后，反馈路径穿过 tL_D 个解码器模块，每个 token 则执行 $L_E + L_D$ 个模块。早期状态的影响取决于门控、投影和状态转移雅可比矩阵的乘积。编码器可并行处理已知 token，但长度为 T 的提示仍需 T 次顺序解码器更新。不同序列的更新可以组成批次，各自维护状态和缓存。附录 B 给出计算量、缓存存储量和提示与回复划分不变性的推导。

2.7 训练与状态重放

训练使用教师强制，并对所选的下一 token 目标计算交叉熵。完整的随时间反向传播 (BPTT) 经过递归输出、解码器 KV 和编码器记忆；掩去某个目标的损失不会跳过对应的状态更新。实验使用的截断 BPTT 间隔覆盖全部训练序列，因此没有切断梯度。在强化学习 (RL) 中，若要在参数更新后评估已采样回复的概率，需用当前参数重放历史并重建状态。附录 D 给出预训练、监督微调 and 策略梯度的目标函数与重放条件。

3 算法任务实验

我们在加法、奇偶校验、模运算和置换状态跟踪任务上比较编码器与解码器的深度分配。六种架构均在六个任务上使用初始化种子 42、43 和 44，共完成 108 次运行，每次训练 2,000 个优化器步骤。我们报告训练长度范围内的验证曲线，以及各次运行分布内最佳检查点的长度泛化结果。所有聚合结果均报告三个初始化种子的均值与样本标准差。

3.1 模型、任务与评估协议

所有模型均有八个逻辑层，宽度为 512，FFN 宽度为 1,365，注意力头数为四。我们将不共享权重的 RLT-1 4 + 4、5 + 3、6 + 2、7 + 1 和 8 + 0 配置，与八层的仅含解码器的 Transformer 比较。RLT-1 使用大小为八的 SWA 窗口、一个共享编码器记忆组，以及反馈幅度 $\alpha = 0.1$ 。8 + 0 变体没

表 1 训练 2,000 个优化器步骤后的验证准确率 (%)。所有条目均为种子 42、43 和 44 的均值 $\pm$ 样本 SD。加法使用教师强制答案 token，形式任务使用最终标签或状态。每次运行已使用 1,024,000 个训练样本。

Task	RLT-1 4+4	RLT-1 5+3	RLT-1 6+2	RLT-1 7+1	RLT-1 8+0	GPT 8
Addition	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
Parity	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	98.83 $\pm$ 1.92	94.84 $\pm$ 3.43
Mod 5, no brackets	45.36 $\pm$ 46.43	94.18 $\pm$ 7.29	69.62 $\pm$ 33.01	70.01 $\pm$ 43.15	60.33 $\pm$ 35.70	64.02 $\pm$ 37.64
Mod 5, brackets	70.53 $\pm$ 10.75	74.35 $\pm$ 6.71	74.87 $\pm$ 3.72	74.78 $\pm$ 3.01	75.17 $\pm$ 6.78	73.87 $\pm$ 9.07
S_5 , swaps	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	99.35 $\pm$ 0.81	99.61 $\pm$ 0.68	99.09 $\pm$ 0.23
S_5 , standard	0.78 $\pm$ 0.68	2.47 $\pm$ 1.26	2.21 $\pm$ 1.48	2.08 $\pm$ 0.98	1.82 $\pm$ 1.19	0.52 $\pm$ 0.23

有解码器模块，但仍执行门控递归融合。RLT-1 的参数量为 26.10–28.73M，Transformer 为 25.31M (表 4)。

所有运行使用全局批量 512、微批量 32，以及 $(\beta_1, \beta_2) = (0.9, 0.95)$ 的 AdamW；权重衰减为 0.1，梯度范数裁剪阈值为 1。基础学习率在前 200 步升至 10^{-4} ，随后按余弦衰减，在第 2,000 步降至 5×10^{-6} 。两类架构采用相同的宽度 μP 配方，以四个线程在 CPU FP32 下运行。RLT-1 的 TBPTT 间隔为 128 个 token，覆盖这些任务的每条训练序列，因此本实验中没有截断梯度。每 100 步执行一次验证。每次运行的完整预算为 1,024,000 个训练样本。

- **加法 (Addition)：**随机采样 1–8 位操作数，将数字按逆序序列化。256 个验证样本使用相同的位数范围。我们测量教师强制答案 token 准确率：每个预测位置都接收正确的先前 token，评分包含答案格式符号与 EOS，排除提示和填充位置。
- **奇偶校验 (Parity)：**使用长度为 3–40 的二进制串训练，只监督最终的奇偶位。验证集汇总长度为 32、36 和 40 的样本，每种长度 256 个。
- **模运算 (Mod-5)：**计算由 +、− 和 $\times$ 构成的表达式模 5 的值。无括号变体遵循乘法优先级，训练表达式长度为 3–39 的奇数，验证长度为 33、35 和 39。带括号变体使用表达式树，训练长度为 3–40，验证长度为 32、36 和 40。每种验证长度包含 256 个样本，仅监督最终值。
- **S_5 状态跟踪：**复合 32 个置换，在每次输入后监督累计乘积，遵循 [Grazzi et al. \(2024\)](#)。标准变体的输入覆盖全部 120 个置换；交换变体使用恒等置换和十个单次对换。每个变体有 256 条长度为 32 的验证序列。主要指标为最终状态准确率，前缀 token 准确率列于附录。

每个任务都使用初始化种子 42、43 和 44，固定数据生成种子、训练样本和验证集。对每次运行，先合并各配置验证长度的计数，再跨初始化求平均。样本标准差 (SD，分母为 $n-1$) 衡量固定数据流上的初始化变异。图 2 在每个优化器步骤上使用全部三个种子；表 1 统一比较第 2,000 步，此时每次运行已使用 1,024,000 个训练样本。

3.2 奇偶校验：学习速度与初始化变异

第 500 步时，RLT-1 6+2 的验证准确率为 $99.44 \pm 0.98\%$ ，Transformer 为 $48.48 \pm 0.53\%$ 。RLT-1 4+4、5+3 和 7+1 的均值约为 83%，SD 为 28–30 个百分点；8+0 仍接近随机水平。6+2 的三个种子均都在第 600 步前（含该步）达到 100%。第 2,000 步时，4+4 至 7+1 均达到 $100 \pm 0\%$ ，8+0 为 $98.83 \pm 1.92\%$ ，Transformer 为 $94.84 \pm 3.43\%$ 。图 3 比较早期与最终检查点。

3.3 带括号与无括号模运算

无括号模运算在第 2,000 步呈现较大的初始化差异。RLT-1 5+3 的均值最高，为 $94.18 \pm 7.29\%$ ，Transformer 为 $64.02 \pm 37.64\%$ 。4+4 的种子 42、43 和 44 分别得到 17.58%、19.53% 和 98.96%，

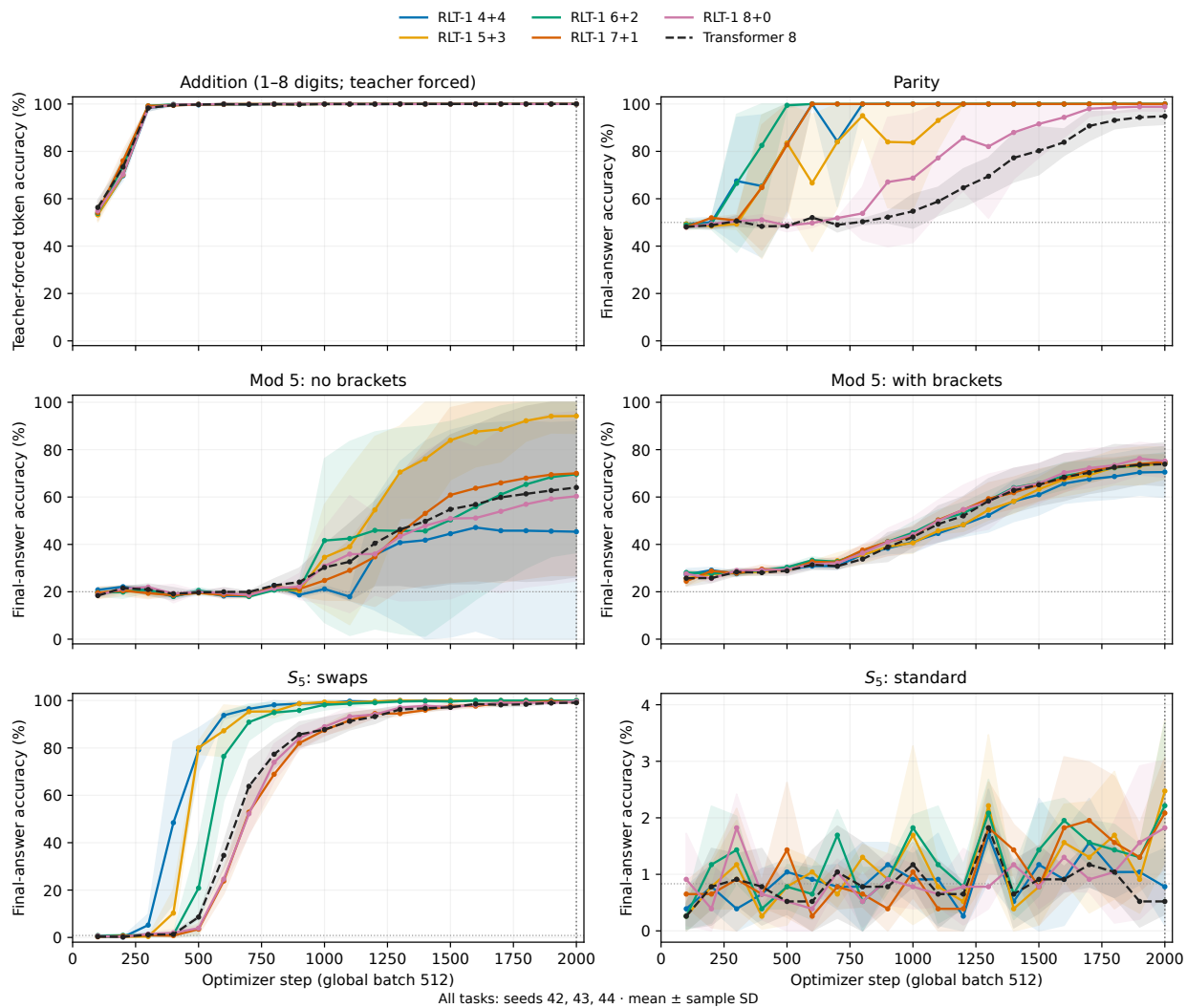


图 2 训练过程中的验证准确率，每个任务均使用三个种子。加法测量教师强制答案 token 准确率，其余任务测量最终标签或最终状态准确率。曲线与阴影表示种子 42、43 和 44 的均值 $\pm$ 样本 SD，每一步均为 $n = 3$ ；阴影带裁剪到准确率范围内。曲线未经平滑，均延伸至第 2,000 步。横向点线表示均匀预测的准确率，标准 S_5 使用较窄的纵轴范围。

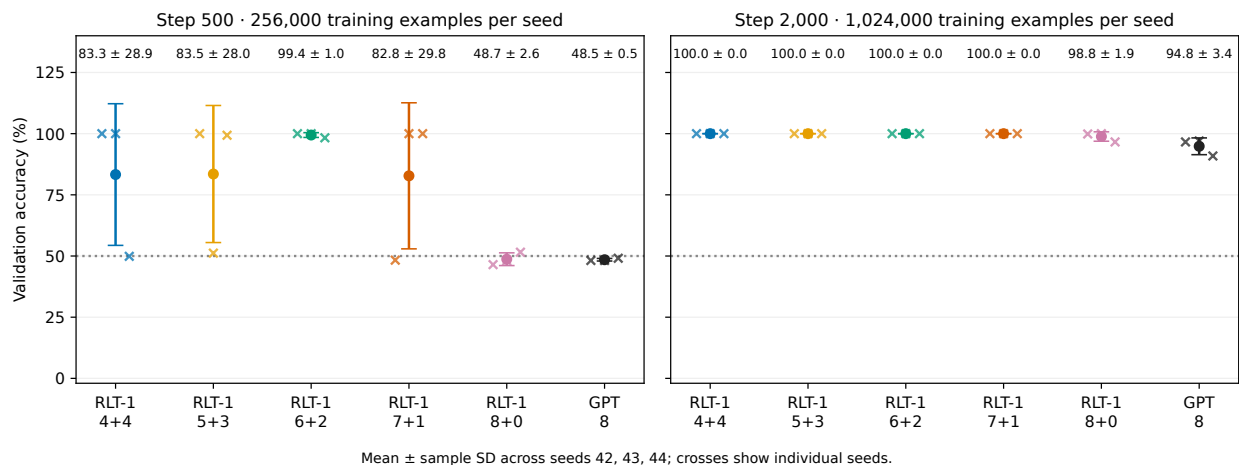


图 3 训练 500 步与 2,000 步后的奇偶校验结果。圆点和误差棒表示种子 42、43 和 44 的均值与样本 SD；叉号表示单个种子，并作少量水平偏移。SD 误差棒可能超过 100%。所有模型使用相同的 768 个验证样本；两个面板分别对应每个种子使用 256,000 和 1,024,000 个训练样本。

均值为 $45.36 \pm 46.43\%$ 。因此，在这一训练预算下，多种模型既有成功学会的运行，也有接近随机水平的运行。

带括号表达式上的模型均值更接近：RLT-1 各配置为 70.53–75.17%，Transformer 为 $73.87 \pm 9.07\%$ 。两种生成器的运算符结构和标签分布不同。括号也占据位置，因此相同的 token 长度可能包含不同数量的算术运算。

3.4 加法与置换状态跟踪

第 2,000 步时，六种架构在混合 1–8 位加法验证集上均达到 $100 \pm 0\%$ 的教师强制 token 准确率。在交换 S_5 上，RLT-1 4 + 4、5 + 3 和 6 + 2 的最终状态准确率为 $100 \pm 0\%$ 。7 + 1 与 8 + 0 分别为 $99.35 \pm 0.81\%$ 和 $99.61 \pm 0.68\%$ ，Transformer 为 $99.09 \pm 0.23\%$ 。

标准 S_5 仍较困难：最终状态准确率的均值为 0.52–2.47%，120 类均匀预测的参考值为 0.83%。前缀 token 准确率的均值为 5.40–7.99%（表 5）；对前缀求平均也包含较容易的早期状态。

3.5 长度泛化

对每个任务、架构和初始化种子，我们从已完成的 2,000 步运行中选取分布内验证样本损失最低的检查点。损失相同时选择最早的检查点，测试结果不参与选择。同一任务的全部 18 个模型与种子组合在各长度上使用相同样本：加法为 256 对操作数，形式任务为 1,024 条序列。图 4 展示完整长度网格，表 2 汇总各任务最长测试长度的结果。附录 J.5 给出检查点步骤和补充指标。

加法与奇偶校验。 操作数超出 1–8 位训练范围后，教师强制加法准确率下降。每个操作数为九位时，RLT-1 7 + 1 为 $68.05 \pm 3.64\%$ ，Transformer 为 $58.61 \pm 2.44\%$ 。到 32 位时，各模型均值处于 14.89–16.84%。附录 J.7 报告独立的十六层实验，同时评估贪心解码的完整答案准确率和教师强制 token 准确率。

即使训练长度上的准确率满分，奇偶校验的长度泛化仍随层数分配而变化。在 256 位时，5 + 3 和 7 + 1 保持 $100 \pm 0\%$ ，6 + 2 为 $84.05 \pm 27.63\%$ ，4 + 4 为 $66.76 \pm 28.78\%$ 。8 + 0 为 $68.91 \pm 27.39\%$ ，Transformer 为接近随机水平的 $50.07 \pm 1.63\%$ 。

表 2 最长测试长度上的准确率 (%)。条目为三个初始化种子的均值 $\pm$ 样本 SD，采用图 4 的检查点与评分规则。加法报告教师强制 token 准确率，形式任务报告最终标签或最终状态准确率。

Task (test length)	RLT-1 4+4	RLT-1 5+3	RLT-1 6+2	RLT-1 7+1	RLT-1 8+0	GPT 8
Addition (32)	15.30 $\pm$ 0.44	14.89 $\pm$ 2.27	15.74 $\pm$ 1.58	15.59 $\pm$ 1.57	15.31 $\pm$ 0.93	16.84 $\pm$ 1.45
Parity (256)	66.76 $\pm$ 28.78	100.00 $\pm$ 0.00	84.05 $\pm$ 27.63	100.00 $\pm$ 0.00	68.91 $\pm$ 27.39	50.07 $\pm$ 1.63
Mod 5, no brackets (255)	18.00 $\pm$ 0.39	20.57 $\pm$ 0.62	21.42 $\pm$ 0.49	19.34 $\pm$ 0.54	20.44 $\pm$ 0.91	20.35 $\pm$ 2.01
Mod 5, brackets (256)	25.20 $\pm$ 1.71	25.81 $\pm$ 4.34	21.58 $\pm$ 1.86	22.04 $\pm$ 1.72	22.30 $\pm$ 1.13	25.07 $\pm$ 2.05
S_5 , standard (512)	1.24 $\pm$ 0.30	1.43 $\pm$ 0.60	1.30 $\pm$ 0.31	0.88 $\pm$ 0.20	0.72 $\pm$ 0.31	0.81 $\pm$ 0.06
S_5 , swaps (512)	55.70 $\pm$ 25.78	34.86 $\pm$ 6.10	22.14 $\pm$ 20.95	0.85 $\pm$ 0.06	0.85 $\pm$ 0.20	0.85 $\pm$ 0.30

模运算。 无括号表达式长度为 63 时, RLT-1 5+3 为 $66.96 \pm 40.04\%$, Transformer 为 $24.48 \pm 3.89\%$ 。长度为 127 时, 5+3 仍有 $39.65 \pm 19.81\%$, Transformer 为 $19.30 \pm 1.04\%$ 。到长度 255 时, 各模型均值为 $18.00\text{--}21.42\%$, 接近 20% 的均匀预测参考值。中间长度上较大的 SD 与无括号模五验证曲线中的训练结果差异一致。带括号模五运算也随表达式变长而降低准确率。

置换状态跟踪。 交换 S_5 在 256 次操作时, RLT-1 4+4 的最终状态准确率为 $97.30 \pm 2.76\%$, Transformer 为 $0.85 \pm 0.30\%$ 。5+3 和 6+2 分别为 $92.71 \pm 1.87\%$ 和 $78.91 \pm 22.26\%$, 7+1 与 8+0 接近均匀预测参考值。在 512 次操作时, 4+4 仍有 $55.70 \pm 25.78\%$ 的最终状态准确率和 $91.16 \pm 6.09\%$ 的前缀 token 准确率; Transformer 分别为 $0.85 \pm 0.30\%$ 和 $9.33 \pm 0.11\%$ 。标准 S_5 在该长度下仍然较低, 最终状态准确率均值为 $0.72\text{--}1.43\%$ 。因此, 较优的层数分配取决于任务: 5+3 和 7+1 能在三个种子上稳定外推奇偶校验, 而分配更多解码器层有助于跟踪更长的交换 S_5 序列。

图 5 比较不同测试长度下的三种状态跟踪准确率。前缀 token 准确率对所有中间状态的预测正确性求平均; 最终状态准确率只评价最后一个状态。全序列准确率要求每个前缀预测都正确, 衡量模型能否在整条序列中持续正确地跟踪状态。最终状态正确并不排除中间预测出错, 而当错误出现在序列后段时, 前缀 token 准确率仍可能较高。

3.6 训练稳定性与比较范围

奇偶校验中, 种子 42 的 RLT-1 5+3 从第 500 步的 100% 降至第 600 步的 48.05%, 随后在第 700 步恢复至 100%。图 15 展示未经平滑的损失, 附录 J 给出奇偶校验的各种子曲线。这次短暂回退和无括号模五运算的种子差异表明, 最终均值可能掩盖训练过程中的不稳定。

与仅含解码器的 Transformer 比较时, 反馈、注意力结构、参数量和计算量同时发生变化。RLT-0 保留编码器与解码器结构及其注意力, 移除可学习初始状态和门控反馈 (图 12); 要分离反馈的作用, 还需进行匹配实验。当前实验评估算法任务的准确率和学习曲线, 未测量硬件吞吐量或 RL 表现。

4 相关工作

Transformer 与 RNN 混合模型。 Chen et al. (2018) 将 Transformer 编码器与基于 LSTM 的 RNMT+ 解码器结合, 用于机器翻译。RLT-1 同样采用并行编码与递归解码的总体分工。其解码器由带编码器记忆交叉注意力和 SWA 的 Transformer 模块组成, 并在因果语言模型中处理提示与回复的全部 token。

编码器生成的记忆。 YOCO 为上层交叉解码器构建可复用的 KV 记忆, 并允许预填充提前退出 (Sun et al., 2024)。DeepSeek-V4.1-Flash 从最终编码器状态投影得到解码器的全局 KV, 并维护独

Length generalization · 8 layers · seeds 42, 43, 44

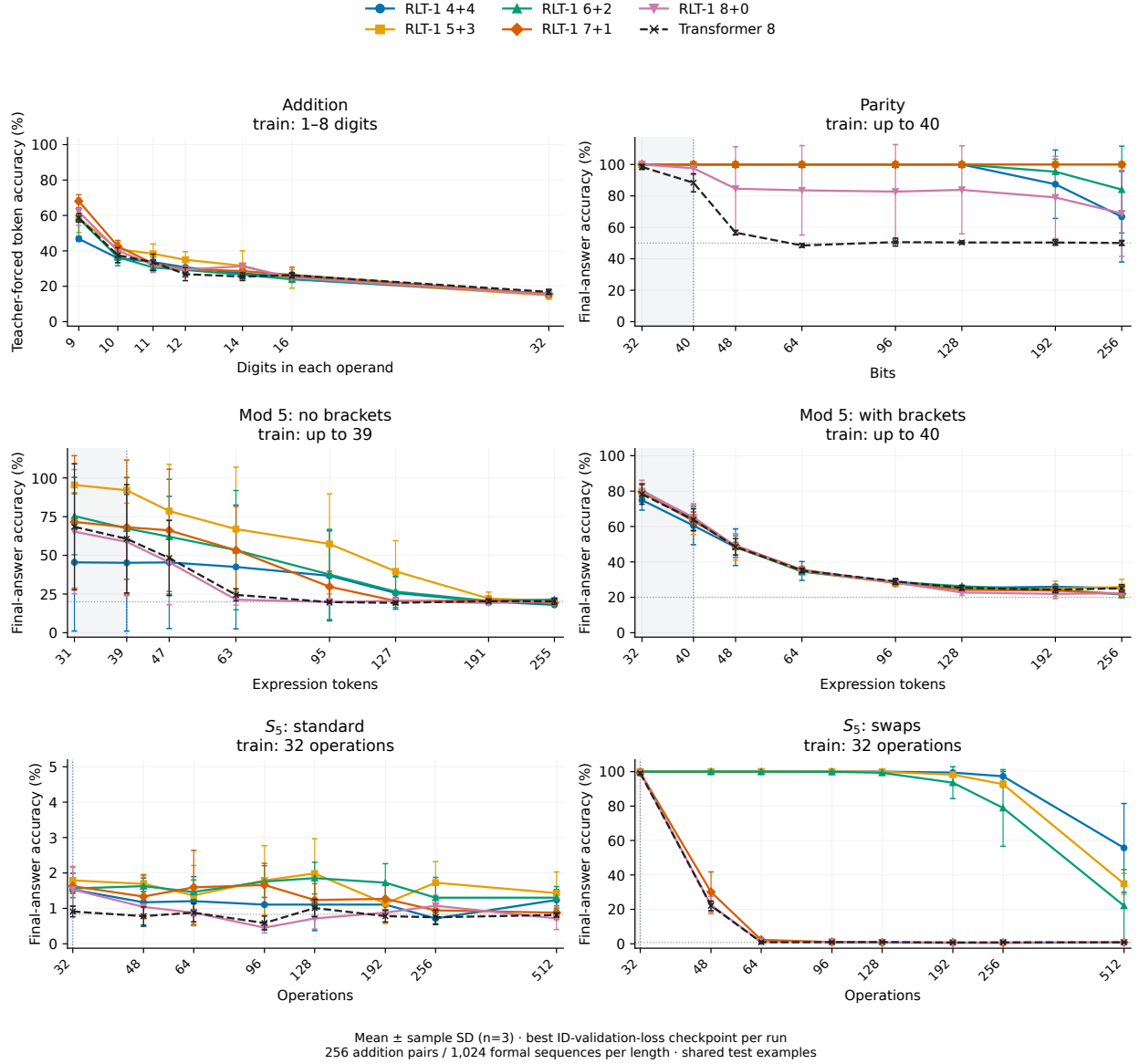


图 4 分布内最佳检查点的长度泛化，三个初始化种子。加法在每种操作数位数的 256 对样本上测量教师强制答案 token 准确率；形式任务在每个长度的 1,024 条序列上测量最终标签或最终状态准确率。点与误差棒表示种子 42、43 和 44 的均值 ± 样本 SD，所有模型使用相同测试样本。SD 误差棒可能超出准确率范围。灰色区域表示训练长度，横向点线表示均匀预测准确率。无括号模五运算使用实际奇数表达式长度，不包含 BOS 和末尾等号。

S₅ length generalization · three scoring rules

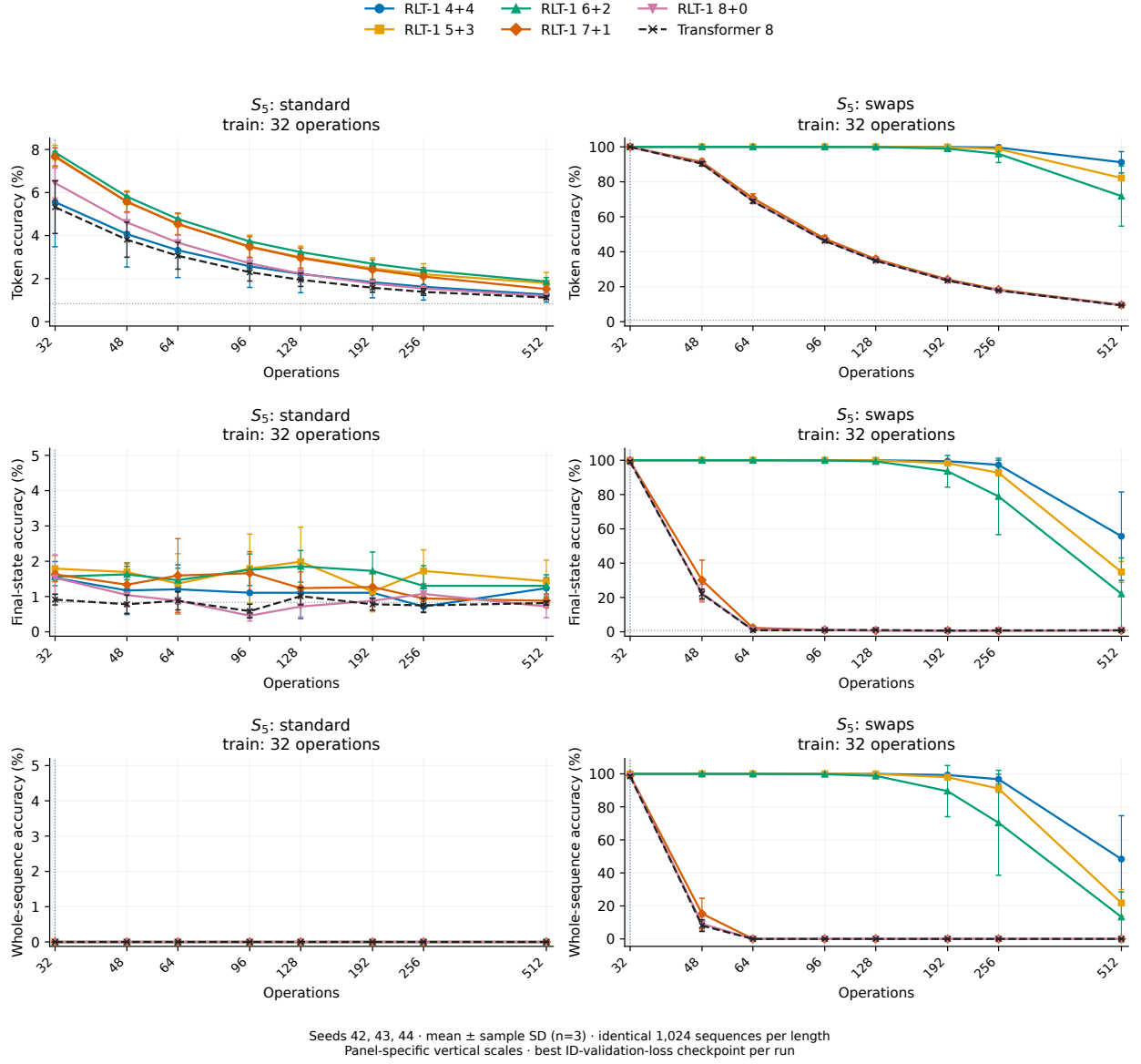


图 5 三种评分规则下的 S₅ 长度泛化。各行依次为前缀 token、最终状态和全序列准确率，各列为标准与交换输入。每个模型与种子在每种长度下接收相同的 1,024 条序列。点和误差棒表示三个初始化种子的均值 ± 样本 SD。全序列成功要求所有前缀预测正确，各面板独立设置纵轴范围以展示标准 S₅ 上较低的准确率。

立的解码器 SWA (DeepSeek-AI, 2026)。RLT-1 也从编码器状态构建全局记忆，但其递归解码器必须处理每个提示 token。

时间反馈。 Feedback Transformer 对每个 token 的各层表示进行可学习的加权求和，并让后续 token 在每一层通过注意力读取这份共享记忆 (Fan et al., 2020)。RLT-1 则在解码器输入处通过门控合并注入前一个最终解码器输出，同时保留独立的编码器全局记忆和逐层解码器 SWA 缓存。Recurrent Transformer 根据每层的输出构建该层的持久 KV，并给出精确的分块调度来改善内存数据搬运 (Oncescu et al., 2026)。RLT-1 的递归依赖贯穿整个解码器，而 Recurrent Transformer 的反馈发生在各层内部。

跨 token 的潜在状态反馈。 Full-bandwidth Transformer 通过门控线性单元，将上一个 token 的顶层隐藏状态与下一个 token 的嵌入结合，同时保留 Transformer 堆栈与 KV 缓存 (Wang et al., 2026)。其多遍训练在各遍之间平移隐藏状态，以支持 token 并行的教师强制训练，并混入前缀计算以处理提示与生成之间的差异。RLT-1 将这种输出到输入的反馈放在编码器与解码器接口处，训练时在完整历史上顺序重放解码器。

T²MLR 将上一个 token 缓存的中间层表示反馈到当前位置的较早层 (Cai et al., 2026)。其实实验发现，中间层之间的递归可能优于贯穿整个网络的递归。训练通过固定次数的 Jacobi 迭代近似时间状态，并单独控制反向传播深度。RLT-1 使用贯穿整个解码器的反馈，以完整历史重放和完整 BPTT 为参考计算，也可选用 TBPTT。

潜在递归语言模型。 Latent Recurrent Transformer (LRT) 通过 KV 投影和残差注入复用上一个 token 的高层状态，保留仅含解码器的主干，并为每个生成 token 执行一次前向计算 (Huang et al., 2026)。其交错并行训练从共享状态缓冲区中读取状态，逐步细化部分位置。在 RL 中，它使用已分离梯度的 rollout 状态初始化该缓冲区，以减小 rollout 与重算之间的差异。RLT-1 将编码器记忆与递归解码器分开，并在当前参数下重建完整历史，以执行精确的策略重放。

连续潜在计算。 Coconut 在潜在推理阶段将最后的隐藏状态作为下一个输入嵌入反馈，并通过课程学习逐渐以连续状态替代文本推理步骤 (Hao et al., 2024)。PonderLM-2 在预训练期间于普通 token 之间插入潜在步骤，并使用 Jacobi 迭代并行近似这些步骤的递归依赖 (Zeng et al., 2025)。RLT-1 对提示或回复中的每个普通 token 更新一次状态，不增加潜在位置。

块级与段级递归。 Block-Recurrent Transformers 通过注意力和门控更新持久状态向量，并在每个递归步骤中并行处理一块 token (Hutchins et al., 2022)。其 block-feedback 变体允许所有层通过交叉注意力读取前一块的递归状态。Recurrent Memory Transformer 在每段末尾添加写入记忆 token，将其最终表示作为下一段的记忆输入，并沿该连接执行 BPTT (Bulatov et al., 2022)。RLT-1 在每个 token 处更新最终解码器输出和 SWA 缓存，同时维护独立的编码器记忆。这种更细的更新间隔要求在每个块内顺序执行解码器。RLT-2 同样在块边界更新反馈，但直接使用最后一个普通 token 的最终解码器输出作为边界状态，并通过门控将其合入下一块的每个解码器输入，无需专门的记忆 token 或独立的递归状态向量组。

跨深度的权重共享。 Universal Transformers 在深度方向共享权重，并可按位置自适应调整表示细化的步数 (Dehghani et al., 2018)。Saunshi et al. (2025) 研究重复应用共享 Transformer 堆栈如何增加推理的有效深度；递归深度语言模型则在推理时调整潜在计算量 (Geiping et al., 2025)。DeepLoop 分析权重共享的循环 Transformer 中重复访问参数对残差缩放的影响，并为 Post-LN 架构推导考虑循环次数的缩放规则 (Li et al., 2026)。RLT-1 跨 token 推进状态，并可选择共享编码器与解码器中相容的权重。

5 结论

RLT-1 将可并行构建的编码器记忆与逐 token 更新的递归解码器结合。在总深度固定为八层时，编码器与解码器之间的层数分配同时影响学习速度和长度泛化。奇偶校验在 $5 + 3$ 和 $7 + 1$ 上表现最稳定，较长的交换置换序列则受益于更深的解码器；加法和模运算的泛化更有限。结果表明，递归计算的深度分配值得作为架构选择单独研究。

参考文献

- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. *arXiv preprint arXiv:2207.06881*, 2022. URL <https://arxiv.org/abs/2207.06881>.
- Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T²MLR: Transformer with temporal middle-layer recurrence. *arXiv preprint arXiv:2607.15178*, 2026. URL <https://arxiv.org/abs/2607.15178>.
- Mia Xu Chen, Orhan Firat, Ankur Bapna, Melvin Johnson, Wolfgang Macherey, George Foster, Llion Jones, Niki Parmar, Mike Schuster, Zhifeng Chen, Yonghui Wu, and Macduff Hughes. The best of both worlds: Combining recent advances in neural machine translation. *arXiv preprint arXiv:1804.09849*, 2018. URL <https://arxiv.org/abs/1804.09849>.
- DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report, DeepSeek-AI, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/resolve/main/DeepSeek_V41_Tech_Report.pdf. Sections 2.2 and 3.2.2; publicly available from the official DeepSeek model repository.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018. URL <https://arxiv.org/abs/1807.03819>.
- Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*, 2020. URL <https://arxiv.org/abs/2002.09402>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025. URL <https://arxiv.org/abs/2502.05171>.
- Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *arXiv preprint arXiv:2411.12537*, 2024. URL <https://arxiv.org/abs/2411.12537>.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. URL <https://arxiv.org/abs/2412.06769>.
- Zeyi Huang, Xuehai He, Liliang Ren, Yiping Wang, Baolin Peng, Hao Cheng, Shuohang Wang, Pengcheng He, Jianfeng Gao, Yong Jae Lee, and Yelong Shen. Latent recurrent transformer: Architecture exploration, training strategies, and scaling behavior. *arXiv preprint arXiv:2605.26797*, 2026. URL <https://arxiv.org/abs/2605.26797>.
- DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-

- recurrent transformers. *arXiv preprint arXiv:2203.07852*, 2022. URL <https://arxiv.org/abs/2203.07852>.
- Shuzhen Li, Yifan Zhang, Jiacheng Guo, Quanquan Gu, and Mengdi Wang. DeepLoop: Depth scaling for looped transformers. *arXiv preprint arXiv:2607.13491*, 2026. URL <https://arxiv.org/abs/2607.13491>.
- Costin-Andrei Oncescu, Depen Morwani, Samy Jelassi, Alexandru Meterez, Mujin Kwun, and Sham Kakade. The recurrent transformer: Greater effective depth and efficient decoding. *arXiv preprint arXiv:2604.21215*, 2026. URL <https://arxiv.org/abs/2604.21215>.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2502.17416>.
- Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*, 2024. URL <https://arxiv.org/abs/2405.05254>.
- Xi Wang, Ziyang Cai, Zheng Zhan, Harry Dong, Ying Fan, Gustavo de Rosa, Tim Pearce, and John Langford. Full-bandwidth transformer. *arXiv preprint arXiv:2608.08888*, 2026. URL <https://arxiv.org/abs/2608.08888>.
- Boyi Zeng, He Li, Shixiang Song, Yixuan Wang, Ziwei He, Xinbing Wang, and Zhouhan Lin. PonderLM-2: Pretraining LLM with latent thoughts in continuous space. *arXiv preprint arXiv:2509.23184*, 2025. URL <https://arxiv.org/abs/2509.23184>.
- Yifan Zhang et al. Reliable RL scaling requires accounting for Prefill–Decode kernel mismatch. Technical report, Pretraining-RL-Science project, August 2026. URL https://github.com/yifanzhang-pro/Pretraining-RL-Science/blob/master/Prefill_Decode_Kernel_Mismatch.pdf. Dated August 6, 2026; revised August 24, 2026.

附录

A 架构与执行细节	15
A.1 编码器与解码器的权重共享	15
B 计算性质	15
B.1 提示与回复的一致性	15
B.2 预填充计算量与顺序深度	15
B.3 递归路径深度	17
C 硬件执行	17
C.1 独立序列的批处理	17
C.2 内存访问与参数复用	17
C.3 训练内存与数值一致性	17
D 训练目标	18
D.1 自回归预训练	18
D.2 监督微调	18
D.3 策略梯度与重放	19
D.4 训练中的梯度与缓存复用	19
E 多轮推理与缓存复用	19
F 执行流程	21
F.1 递归提示预填充	21
F.2 RLT-0	21
F.3 生成与新的外部输入	21
F.4 预训练与监督微调	21
F.5 当前策略的 RL 重放	22
G 因果性与梯度路径	22
H 缓存状态何时可以复用	23
I RLT-2: 分块并行的隐藏状态反馈	23
I.1 分块状态转移与因果掩码	24
I.2 训练与预填充算法	24
I.3 增量生成与不完整 chunk	26
I.4 训练与推理效率	26
J 实验细节	28
J.1 参数量	28
J.2 RLT-0 的实现	28
J.3 种子聚合与指标定义	28
J.4 完整训练快照的复现	30
J.5 长度泛化协议与补充指标	31
J.6 长度为 32 和 33 时的 token 准确率曲线	36
J.7 十六层模型的加法泛化	37

A 架构与执行细节

A.1 编码器与解码器的权重共享

共享权重的参考 RLT-1 令 $L_E = L_D = L$ 。编码器第 ℓ 层的自注意力与解码器第 ℓ 层的 SWA 共享相容的查询、键、值和输出投影矩阵，FFN 也共享权重。编码器使用自身的因果上下文，解码器 SWA 使用窗口内的解码器激活。解码器交叉注意力拥有独立的查询及输出投影，并使用式 (2.2) 的记忆投影。编码器与解码器保留各自的归一化模块，融合与读出也为独立模块。交叉注意力在共享的注意力和 FFN 之外增加计算。

两次计算共享权重，但产生独立的激活。不共享权重的 E_θ, D_ϕ 使用各自的参数，递归形式相同。两种版本都可以共享记忆组；各解码器层仍维护独立的 SWA 缓存。

B 计算性质

B.1 提示与回复的一致性

命题 B.1 (服务划分不变性). 固定一条独立序列的参数、token 序列、位置约定和初始状态。假设采用精确算术，因果编码器的执行在数学上等价，SWA 窗口与缓存更新相同，解码器操作是确定性的。先批量执行编码器预填充、再递归更新解码器，与逐 token 增量处理该前缀，得到相同的状态和下一 token 分布。因此，对于固定的 token 历史，移动提示与回复的划分位置不会改变条件分布。

证明. 因果编码器的等价性保证两种调度具有相同的 e_t 和 $M_{\leq t}$ 。二者均从 $H_0 = (s_\star, \emptyset)$ 开始。若在 $t-1$ 时状态一致，式 (2.5) 在 t 时会对相同输入执行相同操作，因此二者在 t 时的状态仍一致。由于状态转移不依赖服务划分，归纳可得结论。□

B.2 预填充计算量与顺序深度

令 $C_E^{\text{pf}}(T)$ 为编码器预填充计算量， $C_M(T)$ 为记忆投影计算量， $C_D^{\text{step}}(t)$ 为一次解码器计算量：它读取 t 条编码器记忆、每层至多 W 个解码器位置，并包含融合开销。于是

$$C_{\text{prefill}}(T) = C_E^{\text{pf}}(T) + C_M(T) + \sum_{t=1}^T C_D^{\text{step}}(t). \quad (\text{B.1})$$

对于稠密注意力及与模型宽度成比例的 KV 宽度，粗略的算术计算量为

$$O((L_E + L_D)(Td^2 + T^2d) + GTd^2 + L_DT \min(W, T)d). \quad (\text{B.2})$$

预填充需要 T 次顺序执行的解码器状态转移，每次包含 L_D 个模块。即使算术复杂度与稠密 Transformer 同阶，这一依赖也可能降低硬件利用率。

每个新 token 执行 $L_E + L_D$ 个模块，另加融合与记忆投影。注意力计算量仍随上下文长度增长。设有效 KV 宽度为 d_{KV} ，则推理缓存的存储量近似为

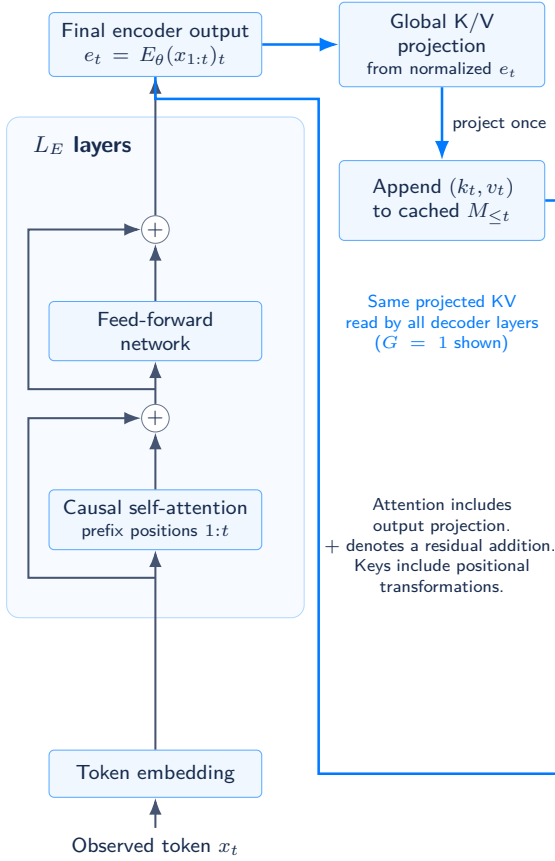
$$O((L_E + G)t d_{\text{KV}} + L_D \min(t, W - 1)d_{\text{KV}}^D + d). \quad (\text{B.3})$$

其中 d_{KV}^D 为解码器 SWA 的有效 KV 宽度， $O(d)$ 项用于保存当前递归输出。SWA 项只计算保留的历史；当前位置的 KV 和训练激活还需要额外存储。权重共享减少参数存储，但两次计算及其缓存仍不可省略。

Causal encoder

Causal prefix $x_{1:t}$

Causal masking allows parallel encoder prefill across positions.



Recurrent decoder

One update at position t

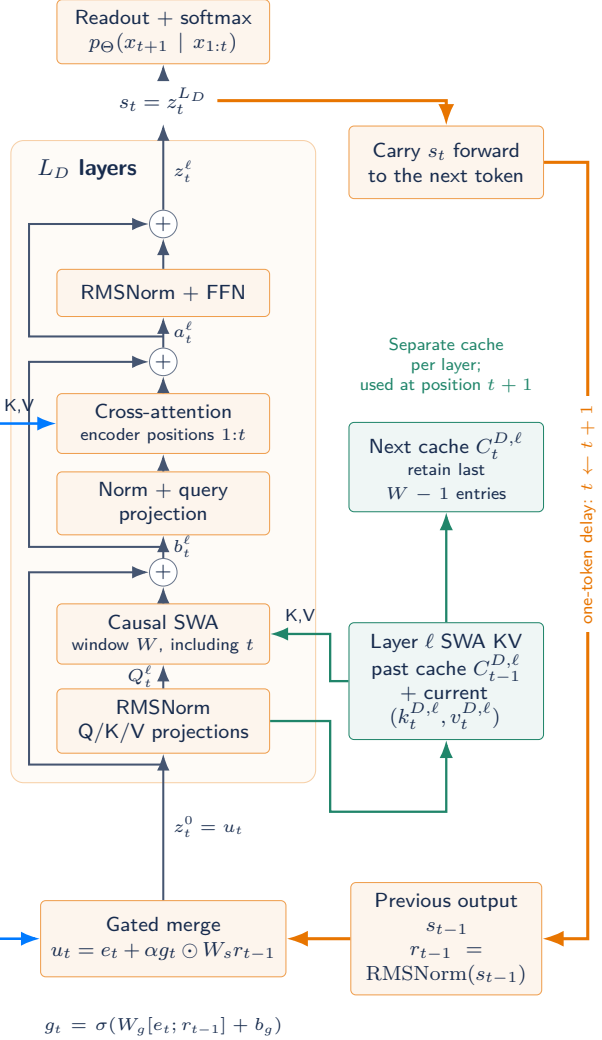


图 6 单次 token 更新的 RLT-1 架构。 计算沿编码器和解码器堆栈向上进行，各虚线框中的模块按标注的深度重复。编码器输出同时用于门控融合和全局 KV 投影。图中取 $G = 1$ ：所有解码器层使用各自的查询，读取同一份已投影并缓存的全局 KV。每个解码器层从自己的输入构建独立的 SWA KV，先关注局部窗口，再读取全局记忆，最后应用 FFN。残差路径绕过各注意力或 FFN 子层。解码器最终输出 s_t 进入下一个 token 的融合操作，各层同时保留自己的 SWA 缓存。蓝色箭头表示编码器特征和全局记忆，绿色表示局部 KV，橙色表示递归隐藏状态。

B.3 递归路径深度

处理长度为 T 的提示和 n 个回复 token 后，递归路径已穿过 $(T + n)L_D$ 个解码器模块。图 7 将这条不断增长的路径与每 token 固定的模块数作比较。早期状态的影响取决于学到的门控、投影，以及状态转移雅可比矩阵的乘积。

Block count per token and along the recurrent path

Illustrative 48-layer encoder + 48-layer decoder; counts exclude merge and readout.

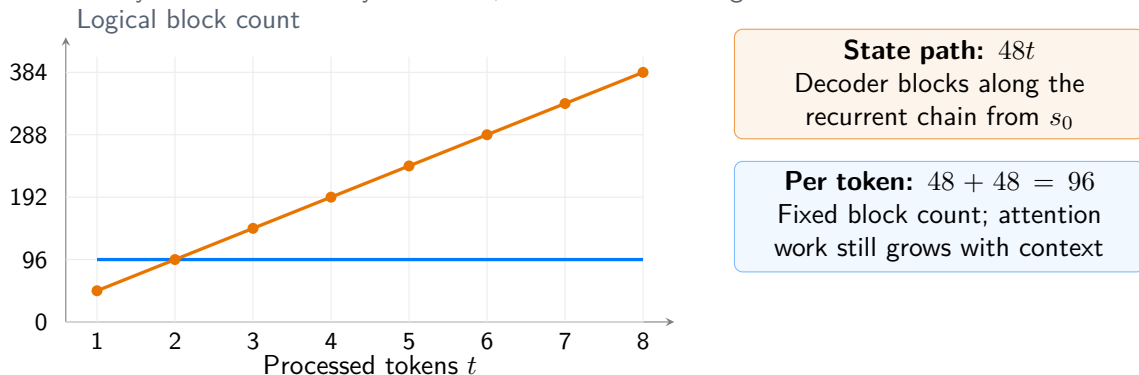


图 7 递归路径长度与每 token 的模块数。当 $L_E = L_D = 48$ 时，处理 t 个 token 后，路径穿过 $48t$ 个解码器模块。每个 token 执行 96 个编码器和解码器模块。橙色统计递归路径上的解码器模块，蓝色统计每个 token 在两个堆栈中执行的模块。

C 硬件执行

C.1 独立序列的批处理

对于已知的训练序列或提示，编码器可跨位置并行计算特征与记忆投影。解码器随后按位置顺序处理。如图 8 所示，不同独立序列的下次解码器更新可以组成同一批次。每条序列分别维护自己的编码器前缀、SWA 缓存、递归输出和位置。

推理时，每一步包含编码器更新、记忆追加、融合、解码器计算和读出。将请求组成批次可增大矩阵运算规模，并复用权重。小批量和不均匀的序列长度可能限制利用率。

C.2 内存访问与参数复用

当前缀和参数不变时，编码器 KV 保持不变。解码器各层复用该记忆，并将自己的 KV 追加到各自的 SWA 缓存中。较少的记忆组可减少 KV 存储和投影计算，较多的组则允许各层使用不同的变换。编码器与解码器共享权重也可能有助于让权重常驻加速器。

一种拟议的优化是融合归一化、门控、状态投影和残差相加，同时保持其依赖关系与数值行为。注意力内核必须保持编码器前缀掩码和局部 SWA 窗口。

C.3 训练内存与数值一致性

激活检查点通过在随时间反向传播 (BPTT) 时重算激活，减少需要保存的激活量。批量大小和检查点位置取决于序列长度及可用的加速器内存。完整 BPTT 保留贯穿整个递归历史的梯度。

Batching decoder updates from independent sequences

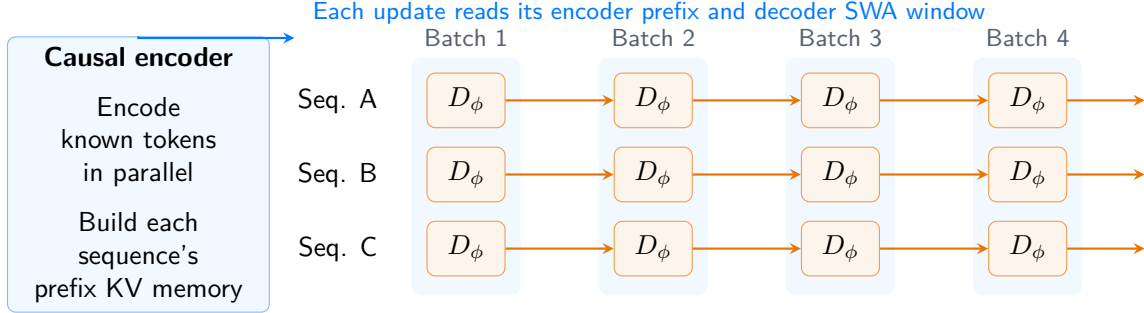


图 8 跨序列批量执行解码器更新。每列将独立更新组成一个批次，每行按 token 顺序展示一条序列。每次更新读取自己的编码器前缀和解码器 SWA 窗口。已知 token 可以在重放前并行编码，生成的 token 则在产生后编码。

Zhang et al. (2026) 指出，实际执行的策略不仅取决于权重，还取决于精度、缓存构造、归约运算和采样变换。因此，采样器与训练器的实现需要保持位置约定、随机行为和缓存内容的一致，并在不同执行模式之间进行数值检查。

D 训练目标

D.1 自回归预训练

基础目标是全序列的下一 token 预测：

$$\mathcal{L}_{\text{PT}}(\Theta) = -\mathbb{E}_{x_{1:S}} \left[\frac{1}{S-1} \sum_{t=1}^{S-1} \log p_{\Theta}(x_{t+1} \mid x_{1:t}) \right]. \quad (\text{D.1})$$

初始化 $H_0 = (s_*, \emptyset)$ ，计算因果编码器特征，并在位置 $1, \dots, S-1$ 上更新解码器。每个非 BOS 目标都计入损失；编码器、记忆投影、融合和解码器联合训练。开始一篇独立文档时，重置解码器与编码器状态及位置，并禁止跨文档边界的注意力。分段训练时，应明确初始上下文与状态：丢弃较早的状态会改变似然所依赖的历史。

D.2 监督微调

对助手目标令 $m_{t+1} = 1$ ，对用户、系统、工具或填充目标令其为 0。对于至少包含一个选中目标的样本，优化

$$\mathcal{L}_{\text{SFT}}(\Theta) = -\mathbb{E}_x \left[\frac{1}{\sum_{t=1}^{S-1} m_{t+1}} \sum_{\substack{1 \leq t < S \\ m_{t+1}=1}} \log p_{\Theta}(x_{t+1} \mid x_{1:t}) \right]. \quad (\text{D.2})$$

该掩码只选择损失项。用户、系统和工具 token 仍更新解码器状态，助手损失的梯度会穿过这些更新，包括编码器记忆与解码器 KV。助手回合开始时，模型从已有状态继续。

D.3 策略梯度与重放

令 $c = x_{1:T}$ 为提示, $y = (y_1, \dots, y_N)$ 为采样回复, 其中 $y_i = x_{T+i}$ 。策略为

$$\pi_{\Theta}(y | c) = \prod_{i=1}^N p_{\Theta}(y_i | c, y_{<i}). \quad (\text{D.3})$$

对于不依赖 Θ 的序列奖励 $R(c, y)$, 定义 $J(\Theta) = \mathbb{E}_{c, y \sim \pi_{\Theta}}[R(c, y)]$ 。其同策略得分函数梯度为

$$\nabla_{\Theta} J = \mathbb{E}_{c, y \sim \pi_{\Theta}} \left[(R(c, y) - b(c)) \sum_{i=1}^N \nabla_{\Theta} \log p_{\Theta}(y_i | c, y_{<i}) \right], \quad (\text{D.4})$$

其中 $b(c)$ 是不依赖回复的基线, 在计算策略梯度时视为常数。重放时, 已采样的 token 固定, 梯度穿过用于计算这些 token 对数概率的递归计算。

图 9 展示重放流程。采样器记录每个动作在实际采样分布 μ 下的对数概率。训练器从初始状态出发, 在当前参数 Θ 下处理完整提示和已采样的回复前缀, 重建编码器特征、递归输出及解码器 SWA 缓存。由所得动作概率构造比率

$$r_i(\Theta) = \exp(\log p_{\Theta}(y_i | c, y_{<i}) - \log \mu(y_i | c, y_{<i})). \quad (\text{D.5})$$

此处目标策略为原始模型策略 p_{Θ} 。行为概率必须包含采样器使用的温度缩放、截断和重新归一化。精确重要性采样要求 $p_{\Theta}(\cdot | h) \ll \mu(\cdot | h)$: 在每个相关历史下, 目标概率为正的動作, 其行为概率也必须为正。Top- k 或 top- p 采样通常会排除未经截断的 softmax 目标中概率为正的動作。如果目标策略本身也使用采样变换, 应在目标函数与比率中一致应用该变换。

训练器必须同时保持前向概率及其对参数的导数 (Zhang et al., 2026)。因此, 它需要对递归计算或其等价实现求导。

对于固定提示和序列奖励, 精确的轨迹重要性采样使用 $\prod_i r_i$, 并需满足支持集与可积性条件。即使重放概率正确, 逐 token 裁剪等替代损失仍可能引入偏差。

D.4 训练中的梯度与缓存复用

完整 BPTT 沿整个提示和回复传播梯度, 路径包括编码器记忆、递归输出和解码器 SWA KV。即使仅回复 token 贡献损失, 这一点也不变。在当前参数下用 `no_grad` 重算提示, 可得到相同的前向值, 但会丢失提示状态的导数。

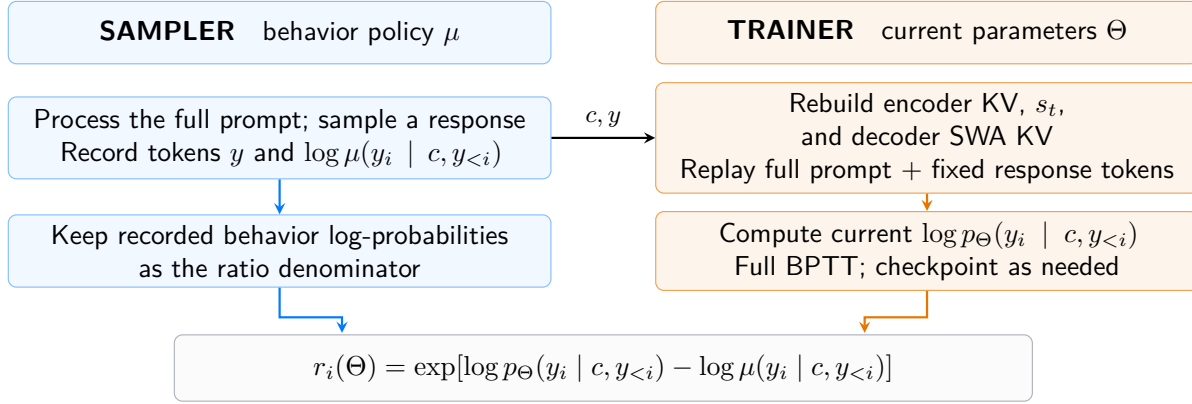
参考计算采用完整 BPTT, 并按需使用激活检查点。检查点机制在反向传播时以相同参数和随机状态重算激活。截断 BPTT (TBPTT) 则在边界处分离选定张量, 保留其数值但切断梯度路径。仅分离 s_t 仍会留下经过解码器 KV 和编码器记忆的路径, 因此截断方案必须说明分离哪些张量。

每次重放及反向传播期间, 参数保持固定。优化器更新后, 在下次精确重放前, 重建编码器 KV、递归输出和解码器 SWA KV。重算可从序列起点开始, 也可从使用当前参数与执行设置生成的精确前缀检查点开始。对于完整 BPTT, 检查点必须在计算图中保留对参数的依赖, 或在反向传播时通过重算前缀重建这些依赖。已记录的行为对数概率保持不变, 因为它们描述的是采样这些动作时的策略。

E 多轮推理与缓存复用

对话构成一条从 BOS 开始的 token 历史。用户消息、工具结果、角色分隔符和助手 token 都执行编码器与解码器更新。例如, 工具返回时, 先结合缓存前缀编码其输出, 再将这些 token 依次送入解码器, 随后恢复生成。仅在开始新的独立序列或明确执行上下文重置时重置状态。

Replaying a sampled history under current parameters



After every parameter update: rebuild current-policy KV and states; retain original behavior log-probabilities.

图 9 在当前参数下重放采样历史。采样器记录行为概率。训练器重建完整前缀，包括递归输出和解码器 SWA 缓存，以评估当前策略。比率比较当前动作概率与记录的行为概率。

若需从保存的前缀继续，应保留编码器缓存 C_t^E 、交叉注意力记忆 $M_{\leq t}$ 、解码器状态 $H_t = (s_t, C_t^D)$ 、token 与位置、SWA 窗口约定以及模型版本。权重固定时，无论提示在哪个位置结束，这些值都可复用。若要复现相同的采样输出，还需要保存采样器的随机状态。如果只保存了文本，应重放文本来重建状态。

编辑或截断较早的 token 会使后续缓存状态失效。应从编辑位置之前的有效检查点重新计算；改写早期 token 的聊天模板也遵循同一规则。在多轮 RL 中，用户和工具 token 更新状态并传递梯度，但不乘动作重要性比率，因为这些 token 并非由策略采样。

F 执行流程

完整的解码器状态为 $H_t = (s_t, C_t^D)$ ，初始值为 $H_0 = (s_*, \emptyset)$ 。编码器的续接状态与编码器生成的交叉注意力记忆分别维护。所有调度均使用式 (2.16) 中的模块顺序和窗口约定。

F.1 递归提示预填充

1. 以 $x_1 = \text{BOS}$ 和 $H_0 = (s_*, \emptyset)$ 开始一条独立序列，初始化编码器缓存与位置。
 2. 使用因果掩码并行编码 $x_{1:T}$ ，构建编码器 KV 记忆。
 3. 对 $t = 1, \dots, T$ ，计算 $H_t = D_\phi(\text{Merge}(e_t, s_{t-1}); M_{\leq t}, C_{t-1}^D, t)$ 。各解码器层读取当前位置的 KV 及保留的 SWA 历史。两种注意力掩码均排除 t 之后的位置。
 4. 从 s_T 预测第一个回复 token。为下一次更新保留 H_T 、编码器缓存、记忆和位置元数据。
- 每次解码器更新产生后续更新所需的 KV，因此即使编码器并行运行，这一循环仍为顺序执行。

F.2 RLIT-0

RLIT-0 移除最终解码器隐藏状态的反馈（图 12）。它保留因果编码器、编码器记忆、解码器 SWA 和限制为前缀的记忆注意力。可学习初始状态 s_* 、状态归一化、门控融合和反馈投影均被移除。每个解码器位置接收 e_t ，代替 $\text{Merge}(e_t, s_{t-1})$ 。

对于已知 token，解码器各位置在每层内使用因果 SWA 和编码器前缀掩码一起运行。所得各层 KV 可保存下来，用于后续增量续接。生成仍按顺序进行，因为未来 token 未知。全序列训练采用普通反向传播，不设置递归 TBPTT 边界。

该对照在保持编码器与解码器分工及注意力结构不变的条件下，检验隐藏状态反馈的作用。

F.3 生成与新的外部输入

从 s_t 预测的分布中采样 x_{t+1} ，随后处理该 token：更新编码器缓存与记忆，并将完整解码器状态推进到 H_{t+1} 。所得状态用于预测 x_{t+2} 。每个被处理的 token 恰好执行一次递归更新，并在每个解码器 SWA 层插入一次 KV。

当用户或工具提供多个 token 时，可在已有前缀条件下，使用因果批处理计算它们的编码器表示。解码器仍从保存的完整状态出发，按 token 顺序更新。外部 token 同时更新 s_t 和 C_t^D 。

若生成因长度上限而停止，且最后一个输出 token 尚未被模型处理，应单独记录该 token。缓存表示已经处理过的前缀；恢复时，先处理一次这个待处理 token，再处理后续 token。

F.4 预训练与监督微调

1. 用因果编码器计算 $e_{1:S-1}$ 并构建记忆。
2. 初始化 $H_0 = (s_*, \emptyset)$ ，展开位置 $1, \dots, S-1$ 的全部计算，包括各层 SWA 缓存更新。
3. 预训练累积所有有效的下一 token 损失，SFT 仅累积助手目标的损失。所有上下文 token 都执行状态更新。
4. 按每个样本选中的目标数量归一化，再按式 (D.1) 和式 (D.2) 对样本求平均。对完整计算反向传播，并按需使用检查点。

没有选中目标的样本不计入损失均值。若改为按 token 求平均，选中目标较多的样本会得到更高权重。

打包在一起的独立序列需要各自的递归状态、SWA 缓存、编码器缓存、位置和注意力掩码。即使损失被掩码屏蔽，两种注意力机制也必须限制在各自文档边界内。训练中的缓存操作必须保留梯度依赖，或支持等价的重算。

F.5 当前策略的 RL 重放

1. 读取 rollout 的 token 历史、动作掩码、实际行为对数概率与采样元数据。行为概率包含全部采样变换和重新归一化。
2. 在前向重放和反向传播期间固定当前参数值。采用目标策略的位置、SWA 和执行约定，并启用参数梯度。
3. 重算已知历史的编码器表示。从 $H_0 = (s_*, \emptyset)$ 开始，经过全部提示 token，重建递归输出与每个解码器 SWA 缓存。
4. 按顺序重放后续 token。在处理每个已采样动作之前，从前一状态读取该动作在当前策略下的对数概率。若 EOS 被采样，也应计入。处理夹在动作之间、后续预测所需的全部外部 token。
5. 根据动作对数概率、奖励或优势，以及所需的行为比率，构造选定的 RL 损失。动作因子仅作用于策略采样的 token。
6. 反向传播并更新参数；在下次精确的当前策略重放前，使依赖参数的编码器和解码器缓存失效。

动作掩码选择损失项，同时保留经过用户与工具 token 的梯度。重放全部助手回合及其间的外部上下文。在 `no_grad` 下重放提示，或分离提示 KV，可保持前向值不变，但会丢失完整 BPTT 所需的梯度。

重要性权重必须满足式 (D.5) 所述的支持集条件。

确定性的前向计算（例如关闭 dropout）可为每个动作给出可复现的概率。对于随机策略，应说明动作概率是以内部随机性为条件，还是对该随机性取平均。一次重放只评估某个具体实现；边缘概率需要对该随机性求平均。

G 因果性与梯度路径

命题 G.1 (因果性). 在确定性执行下，若编码器是因果的，编码器记忆限制为当前前缀，且解码器 SWA 是因果的，则 $H_t = (s_t, C_t^D)$ 仅依赖 $x_{1:t}$ 和参数（包括 s_* ）。

证明. 编码器的因果性意味着各 e_j 仅依赖 $x_{1:j}$ ，因此 $M_{\leq t}$ 仅依赖 $x_{1:t}$ 。初始状态不包含未来 token 的信息。假设 H_{t-1} 仅依赖 $x_{1:t-1}$ 。下次更新使用该状态、 e_t 和 $M_{\leq t}$ 。在每个解码器层，当前位置的 KV 由因果可用的层输入形成，SWA 不读取大于 t 的位置。追加当前 KV 和淘汰旧条目不会引入未来信息。因此， s_t 和 C_t^D 均仅依赖 $x_{1:t}$ ，归纳完成。□

固定教师强制得到的编码器特征，将解码器缓存表示为固定槽位，并在尚未填满时使用有效性掩码，定义

$$\mathcal{J}_t = \frac{\partial H_t}{\partial H_{t-1}}. \quad (\text{G.1})$$

则

$$\frac{\partial H_t}{\partial H_j} = \mathcal{J}_t \mathcal{J}_{t-1} \cdots \mathcal{J}_{j+1}, \quad j < t, \quad (\text{G.2})$$

其中

$$\mathcal{J}_t = \begin{pmatrix} \frac{\partial s_t}{\partial s_{t-1}} & \frac{\partial s_t}{\partial C_{t-1}^D} \\ \frac{\partial C_t^D}{\partial s_{t-1}} & \frac{\partial C_t^D}{\partial C_{t-1}^D} \end{pmatrix}. \quad (\text{G.3})$$

这个分块雅可比矩阵同时包含经过递归输出与解码器 KV 的路径。

编码器记忆保留过去编码器输出的信息；解码器 SWA 则可直接访问近期的解码器投影。较旧的解码器条目会被淘汰，但它们仍可通过淘汰前读取过它们的状态或激活影响后续计算。

完整的参数导数还包含编码器特征、记忆、各次状态转移和解码器 KV 投影对参数的依赖。令 B_T^E 表示回复续接所需的全部编码器侧边界张量，包括编码器续接 KV 和编码器生成的交叉注意力记忆。对于回复损失 $\ell(\Theta, H_T(\Theta), B_T^E(\Theta))$ ，链式法则给出

$$\frac{d\ell}{d\Theta} = \frac{\partial \ell}{\partial \Theta} + \frac{\partial \ell}{\partial H_T} \frac{\partial H_T}{\partial \Theta} + \frac{\partial \ell}{\partial B_T^E} \frac{\partial B_T^E}{\partial \Theta}. \quad (\text{G.4})$$

第一项将边界参数视为固定值，其余项计入这些边界张量的前缀计算。特别地，

$$\frac{\partial \ell}{\partial H_T} \frac{\partial H_T}{\partial \Theta} = \frac{\partial \ell}{\partial s_T} \frac{\partial s_T}{\partial \Theta} + \frac{\partial \ell}{\partial C_T^D} \frac{\partial C_T^D}{\partial \Theta}. \quad (\text{G.5})$$

即使前向概率不变，分离 s_T 、解码器 KV 或编码器边界张量，也会从完整梯度中移除相应项。

H 缓存状态何时可以复用

当保存的前缀与当前参数、已处理 token、位置、初始化、SWA 窗口及策略执行设置一致时，可以用它替代前向重放。需要保存递归输出、每个解码器 SWA 缓存、编码器续接状态、编码器记忆及相关元数据。随机计算按策略的规定处理。

已经分离梯度的缓存可能具有正确数值，却不包含训练所需的计算图。完整 BPTT 需要保留或重算前缀计算图，使梯度能够到达每个依赖参数的边界张量。

一次更新内的激活检查点以相同参数和随机状态重算激活。可变缓存的实现必须在重算时恢复保存的内容，并保留反向传播仍需使用的激活。

截断 BPTT 则保留数值，同时切断选定的梯度依赖。完整解码器状态的分离为

$$\tilde{H}_t = (\text{stopgrad}(s_t), \text{stopgrad}(C_t^D)). \quad (\text{H.1})$$

仅分离 s_t 会留下可能经过解码器 KV 的路径；仅分离解码器 KV 则会留下经过递归输出的路径。编码器侧边界张量还可能提供跨越同一边界的其他路径。

SWA 淘汰使旧 KV 不再进入未来的注意力窗口。完整 BPTT 仍会对之前使用过这些条目的计算求导，因此训练所需的激活存储可能超过推理缓存本身。

权重更新后，应在下一次精确训练计算之前，重算前缀数值及其梯度图。

I RLT-2：分块并行的隐藏状态反馈

RLT-2 在一个 chunk 内保持反馈状态不变，在 chunk 边界更新该状态。因此，chunk 内的已知 token 可以在每个解码器层中并行计算。因果编码器、解码器 SWA、前缀受限的编码器记忆注意力、门控融合和输出头均采用 RLT-1 的组件。本节给出拟议架构及其执行成本；第 3 节的实验评估 RLT-1 和 Transformer。

I.1 分块状态转移与因果掩码

固定 chunk 大小 $B \geq 1$ ，将 BOS 计为位置一，并以每条独立序列的起点为边界基准。对于长度为 T 的已知前缀，定义

$$K = \lceil T/B \rceil, \quad a_k = (k-1)B + 1, \quad b_k = \min(kB, T), \quad I_k = \{a_k, \dots, b_k\}. \quad (\text{I.1})$$

初始化反馈寄存状态 $h_0 = s_*$ 和空的解码器缓存。第 k 个 chunk 的所有位置读取同一个前序边界状态，门控仍依赖各自的位置：

$$r_{k-1} = \text{RMSNorm}_s(h_{k-1}), \quad v_{k-1} = W_s r_{k-1}, \quad (\text{I.2})$$

$$g_t = \sigma(W_g[e_t; r_{k-1}] + b_g), \quad (\text{I.3})$$

$$z_t^0 = e_t + \alpha g_t \odot v_{k-1}, \quad t \in I_k, \quad (\text{I.4})$$

$$y_{I_k}, C_{b_k}^D = D_\phi^{\text{chunk}}(z_{I_k}^0; M, C_{a_k-1}^D, I_k), \quad (\text{I.5})$$

$$p_\Theta(x_{t+1} \mid x_{1:t}) = \text{softmax}(W_o \text{RMSNorm}_o(y_t))_{x_{t+1}}, \quad (\text{I.6})$$

$$h_k = y_{kB} \quad \text{当 } kB \leq T. \quad (\text{I.7})$$

$y_t = z_t^{LD}$ 是每个位置的最终解码器输出；只有完整 chunk 的末位置输出成为下一个反馈状态。分块算子逐层应用式 (2.16)，每层同时处理 I_k 内所有已知位置。查询 t 仅访问 $\max(1, t-W+1) \leq j \leq t$ 的解码器键，包括之前 chunk 保留的键，并且只访问 $j \leq t$ 的编码器记忆。SWA 和编码器位置均不在 chunk 边界重置，chunk 大小也不必等于 SWA 窗口。

每层先根据该层输入并行投影所有位置的 KV，再执行带掩码的注意力，因此层内无需逐位置串行循环。 h_{k-1} 仅依赖 a_k 之前的 token；因果编码器和解码器掩码进一步保证 y_t 仅依赖 $x_{1:t}$ 。末位置输出通过这些因果计算汇总 chunk，不增加池化操作或摘要 token。当 $B = 1$ 且权重与执行约定相同时，该状态转移还原为 RLT-1。即使整条序列只占一个 chunk，仍存在可学习初始状态的融合；RLT-0 则移除了该融合及其参数。

I.2 训练与预填充算法

对于 teacher forcing 训练， $x_{1:T}$ 表示已消费的输入 token，在有后续目标的位置计算 next-token 损失。提示预填充和已知历史的策略重放使用同一前向过程：

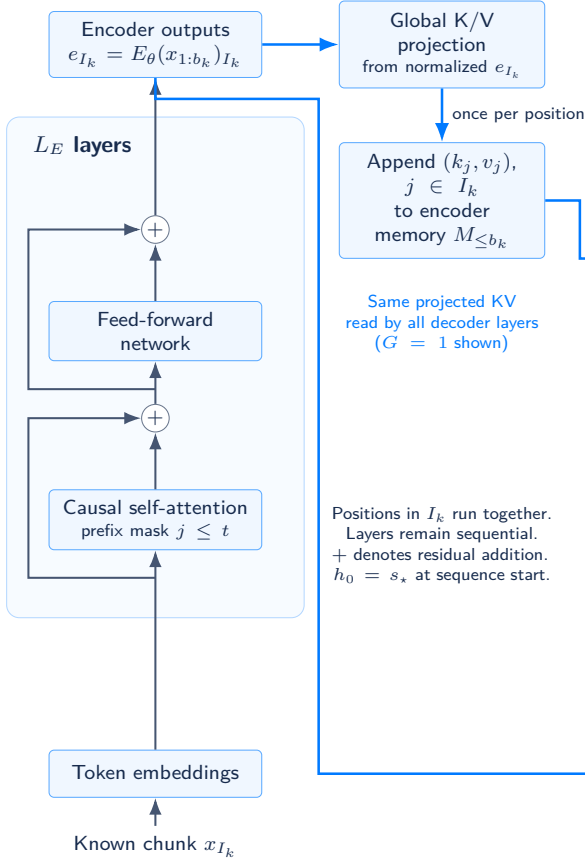
1. 对已知 token 做因果编码并构建编码器 KV 记忆。初始化 $h = s_*$ 、空解码器 SWA 缓存和全局位置一。
2. 按顺序处理 chunk。每个 chunk 只计算一次 $\text{RMSNorm}_s(h)$ 和 $W_s \text{RMSNorm}_s(h)$ ，然后广播到各位置的门控融合。
3. 在每个解码器层中，并行投影 chunk 的 Q/K/V，结合历史和当前 chunk 的 KV 执行因果 SWA，然后执行前缀记忆注意力与 FFN。每层保留最后 $W-1$ 个解码器 KV 条目用于继续计算。
4. 在每个位置计算 logits 和所选的 next-token 损失。完整 chunk 结束后，将 h 更新为其末位置输出；末尾不完整 chunk 不更新 h 。
5. 训练时，累积序列损失并通过全部 chunk 反向传播，再更新参数。预填充时，保存缓存、 h 、已消费 token 数，以及末位置输出或 next-token logits。

完整 BPTT 通过边界隐藏状态、跨 chunk 的解码器 KV 和编码器记忆传播梯度。分块改变前向依赖图，不自动截断梯度，也不表示每个 chunk 更新一次参数。可选的 TBPTT 必须明确截断哪些路径。RL 重放使用与采样相同的 B 、边界基准、掩码和位置，并在当前参数下重建状态后计算动作概率。

Causal encoder

Known prefix through b_k

Causal masking allows parallel encoder prefill across positions.



RLT-2 decoder

Chunk I_k : parallel positions within each layer

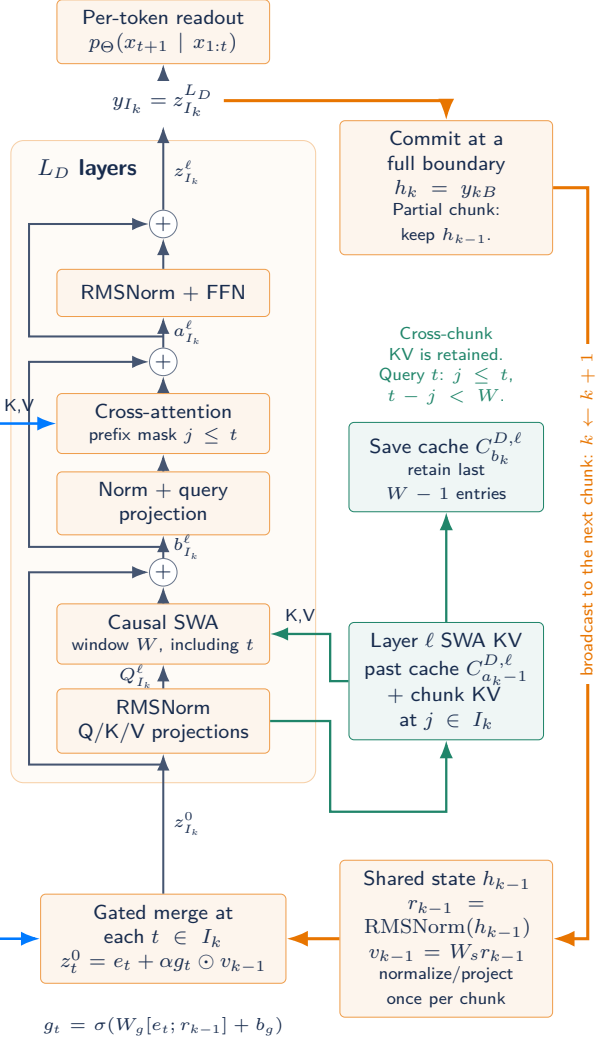


图 10 RLT-2 单个 chunk 的架构。计算自下向上进行，编码器和解码器堆栈分别重复指定层数。各位置 $t \in I_k$ 使用自己的门控，将编码器输出与同一个前序边界状态 h_{k-1} 融合。状态归一化和投影在 chunk 内共享。每个解码器层依次执行因果 SWA、前缀受限的编码器记忆注意力和 FFN，各子层均有残差连接。 $G = 1$ 时所有解码器层读取同一组投影后的编码器 KV，各层分别跨 chunk 边界保留自己的 SWA KV。每个输出 y_t 都预测下一个 token，只有完整边界的 y_{kB} 成为下一个反馈状态。不完整 chunk 在续接时保留 h_{k-1} 。蓝色箭头表示编码器特征和记忆，绿色表示解码器 KV，橙色表示 chunk 级反馈。

I.3 增量生成与不完整 chunk

设已消费 token 数为 t ，当前 chunk 偏移为 $r = t \bmod B$ 。保存的反馈状态为 $h_{[t/B]}$ ，即使提示结束在 chunk 中间也如此。消费下一个已知或采样 token（位置 $j = t + 1$ ）时：

1. 更新因果编码器并追加其记忆 KV。将 e_j 与保存的边界状态融合；该状态在当前 chunk 内保持不变。
2. 使用正常的因果 SWA 缓存和前缀记忆运行一次解码器，得到 y_j 、更新后的 KV 缓存和 next-token logits。
3. 若 $j \bmod B = 0$ ，则以 y_j 更新反馈状态；否则保留原值。将已消费 token 数更新为 j 。

最新 y_j 始终用于下一个 token 的预测，不要求它已写入反馈状态。例如 $B = 4$ 、提示长六个 token 时，位置 5–8 都与 $h_1 = y_4$ 融合；预填充处理位置 5、6 不会用 y_6 替换该状态。 y_6 预测 token 7；只有消费 token 8 后， y_8 才成为 h_2 。已知后续 token 可并行处理到下一个固定边界，再使用新状态继续。Padding、消息边界及提示与回复的分界都不提交不完整 chunk。打包的独立序列分别维护边界基准和状态。

固定权重并采用精确算术时，若两种调度使用相同的数学算子，且关闭随机操作或为对应操作分配相同的随机数，则批量和增量执行等价。此时，每个位置读取相同的编码器前缀、前序边界状态及各层因果 KV。先对 chunk 内各层、再对 chunk 归纳，可得两种执行方式的输出和边界状态相同。改变 B 或边界基准会改变模型计算，使已保存的续接状态失效。

分块预填充与逐 token 解码可能使用不同的矩阵和注意力 kernel，浮点归约顺序也可能不同。即使执行具有确定性，这些差异也可能改变 logits 和边界状态。对于 RL，应将分块重放与复现 rollout 编码器、解码器和输出头执行方式的逐 token 参考路径比较，包括精度、批次布局、kernel 选择及随机设置。应先报告参数不变时所选动作的对数概率差异，再将重放差异归因于策略更新。

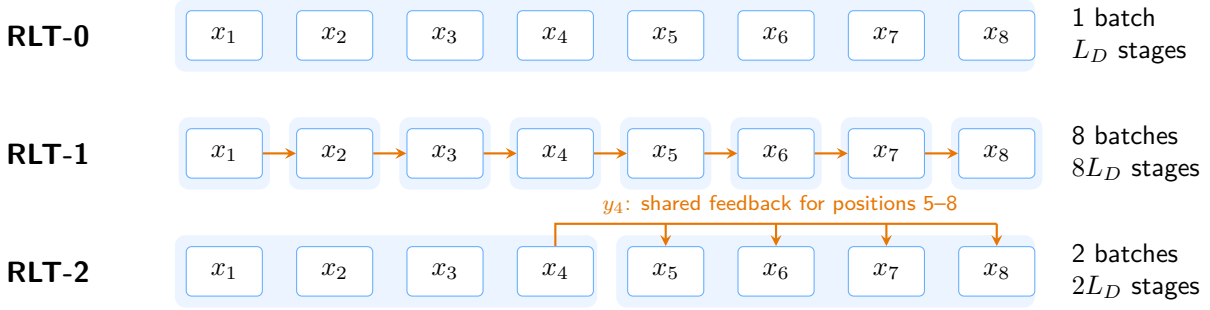
I.4 训练与推理效率

表 3 在相同编码器与解码器深度、宽度、窗口和记忆组数下比较三个变体，并假设 $L_D \geq 1$ 。 T 是已知输入 token 数， $K = \lceil T/B \rceil$ ， N 是生成时新消费的 token 数。解码器模块阶段数表示所述逐层调度中的串行依赖，不包含注意力核内部归约、通信和硬件相关延迟。三个变体均需公共的 L_E 阶段因果编码器预填充。

指标	RLT-0	RLT-1	RLT-2 (B)
已知 token 的每批解码位置数	T	1	至多 B
训练前向 / 预填充的解码模块阶段数	L_D	TL_D	KL_D
完整 BPTT 的反向依赖深度	$O(L_D)$	$O(TL_D)$	$O(KL_D)$
处理 T 个 token 的反馈状态更新数	0	T	$\lceil T/B \rceil$
处理 T 个 token 的解码模块计算数	TL_D	TL_D	TL_D
处理 T 个 token 的融合门控计算数	0	T	T
已知 token 的共享反馈投影数	0	T	K
生成 N 个 token 的自回归步骤数	N	N	N
每个新消费 token 的模块计算数	$L_E + L_D$	$L_E + L_D$	$L_E + L_D$
持久反馈状态	无	d 个值	d 个值

表 3 匹配网络尺寸时的执行成本。省略公共的输出头及编码器记忆投影。RLT-2 缓存归一化与投影后的边界状态，而门控仍依赖每个 token。反向深度统计解码器依赖图，编码器反向传播为公共部分。RLT-0 还省去可学习初始状态和融合参数。这些计数描述依赖关系和算术计算量；实际运行时间取决于实现和硬件。

Known-token decoder batches



Within a batch: positions in parallel at each layer. Between batches: preserve causal SWA caches.

图 11 八个已知 token 的解码调度，RLT-2 取 $B = 4$ 。每个阴影组为一个逐层解码批次，包含 L_D 个串行模块。RLT-0 一批处理八个位置，RLT-1 使用八个单位位置批次，RLT-2 使用两个四位置批次。橙色箭头表示反馈依赖，RLT-2 将 y_4 广播到第二个 chunk 的全部四个位置。各行均使用因果 SWA，并在批次间保留缓存。自回归生成时，三个变体均逐 token 消费未知的未来输入。

固定网络尺寸并采用稠密编码器与记忆注意力时，公共前向算术复杂度为

$$O((L_E + L_D)Td^2 + (L_E + L_D)T^2d + GTd^2 + L_DT \min(W, T)d). \quad (\text{I.8})$$

RLT-1 额外需要 $O(Td^2)$ 的融合计算。RLT-2 保留 T 次位置相关门控，因此融合计算仍为 $O(Td^2)$ ，但每个 chunk 内复用状态归一化和投影。它使用 K 个串行解码批次，每批至多 B 个位置，可增大矩阵计算规模并复权重。总解码算术量仍覆盖全部 T 个 token，完整 BPTT 仍需保存或重算所有 chunk 的激活。

在上下文长度为 t 的生成阶段，各变体每个 token 都需运行编码器与解码器并读取可用编码器记忆。RLT-2 在下一边界前可以复用反馈投影；位置相关门控、解码器模块和 KV 写入仍逐 token 执行。公共推理缓存规模为

$$O((L_E + G)t d_{KV} + L_D \min(t, W - 1)d_{KV}^D). \quad (\text{I.9})$$

RLT-1 和 RLT-2 额外保存 $O(d)$ 反馈状态；RLT-2 还保存 chunk 偏移，并可缓存 $O(d)$ 的归一化和投影状态。分块预填充需要临时激活与当前 chunk 的 KV。增大 B 将反馈路径从 T 次 token 更新缩短为 K 次 chunk 计算，以反馈频率换取已知 token 的并行性。

匹配实验应扫描 $B \in \{1, 2, 4, 8, 16, 32\}$ 并加入 RLT-0，固定数据、网络尺寸、精度、优化器、批量大小和注意力核。同时报告任务准确率、训练 tokens/s（含前向、反向和优化器）、不同提示长度的预填充延迟、不同缓存上下文长度的每 token 生成时间，以及设备峰值内存。采用设备同步与预热，报告硬件及批量大小，并分开测量预填充与生成。 $B = 1$ 检查与 RLT-1 的等价性，扰动未来 token 检查因果性。在参数不变时，测量分块、不完整 chunk 和逐 token 执行之间 logits、所选动作对数概率及边界状态的最大差异和均方根差异，并记录数值精度与执行设置。可微参考实现还应比较完整 BPTT 的参数梯度，包括经过边界状态、解码器 KV 和编码器记忆的路径。

J 实验细节

J.1 参数量

表 4 列出不重复计数的可训练参数量。每个 RLT-1 解码器模块同时具有 SWA 和编码器记忆交叉注意力；每个 Transformer 模块只有一个注意力子层。RLT-1 8+0 在每个 token 处应用可学习的状态投影和门控融合。

表 4 八层模型的不重复可训练参数量。同一模型在六个任务中的规模相同。

Model	Parameters (M)	Relative to GPT 8
RLT-1 4+4	28.73	1.135×
RLT-1 5+3	28.20	1.114×
RLT-1 6+2	27.68	1.093×
RLT-1 7+1	27.15	1.073×
RLT-1 8+0	26.10	1.031×
Transformer 8	25.31	1.000×

J.2 RLT-0 的实现

rlt_parallel_tiny 对照由 [nanogptpro-dev PR #164](#) 合并。它采用不共享权重的 4+4 编码器与解码器分配、宽度 512、四个注意力头、FFN 宽度 1,365、词表容量 277、大小为八的 SWA 窗口、一个编码器记忆组，以及宽度 μP 。它共有 27,938,944 个不重复参数。

该对照从 RLT-1 4+4 中移除可学习初始状态、状态归一化、融合门控和反馈投影。它保留因果编码器注意力、解码器 SWA，以及仅关注前缀的编码器记忆注意力。已知位置在每个解码器层内并行运行，训练采用完整反向传播，不使用 TBPTT。生成阶段维护 KV 缓存并按顺序输出 token，不把前一个最终隐藏状态反馈到下一个解码器输入。两个模型使用由各自架构标识区分的独立检查点格式。

J.3 种子聚合与指标定义

令 $a_{m,s,t}$ 为模型 m 、初始化种子 s 在优化器步骤 t 的验证准确率。对每个任务均报告

$$\bar{a}_{m,t} = \frac{1}{3} \sum_{s \in \{42,43,44\}} a_{m,s,t}, \quad SD_{m,t} = \sqrt{\frac{1}{2} \sum_{s \in \{42,43,44\}} (a_{m,s,t} - \bar{a}_{m,t})^2}. \quad (J.1)$$

每个聚合结果使用同一步的全部三个种子，不进行插值或缺失种子填补。所有曲线延伸至第 2,000 步，表 1 统一使用该步。SD 描述固定训练数据流与留出集上不同模型初始化之间的变异。

奇偶校验和模五运算每个样本监督一个最终标签。先合并各次运行中三个大小相同的验证组，再跨种子求平均。 S_5 监督每个前缀；最终状态准确率只统计每条序列的最后一个状态，前缀 token 准确率合并全部受监督位置。加法使用完整提示和正确的先前答案 token 进行教师强制。其分母包含数字、空格、括号与 EOS 等全部答案 token，排除提示与填充位置。加法损失是按样本归一化的教师强制交叉熵。

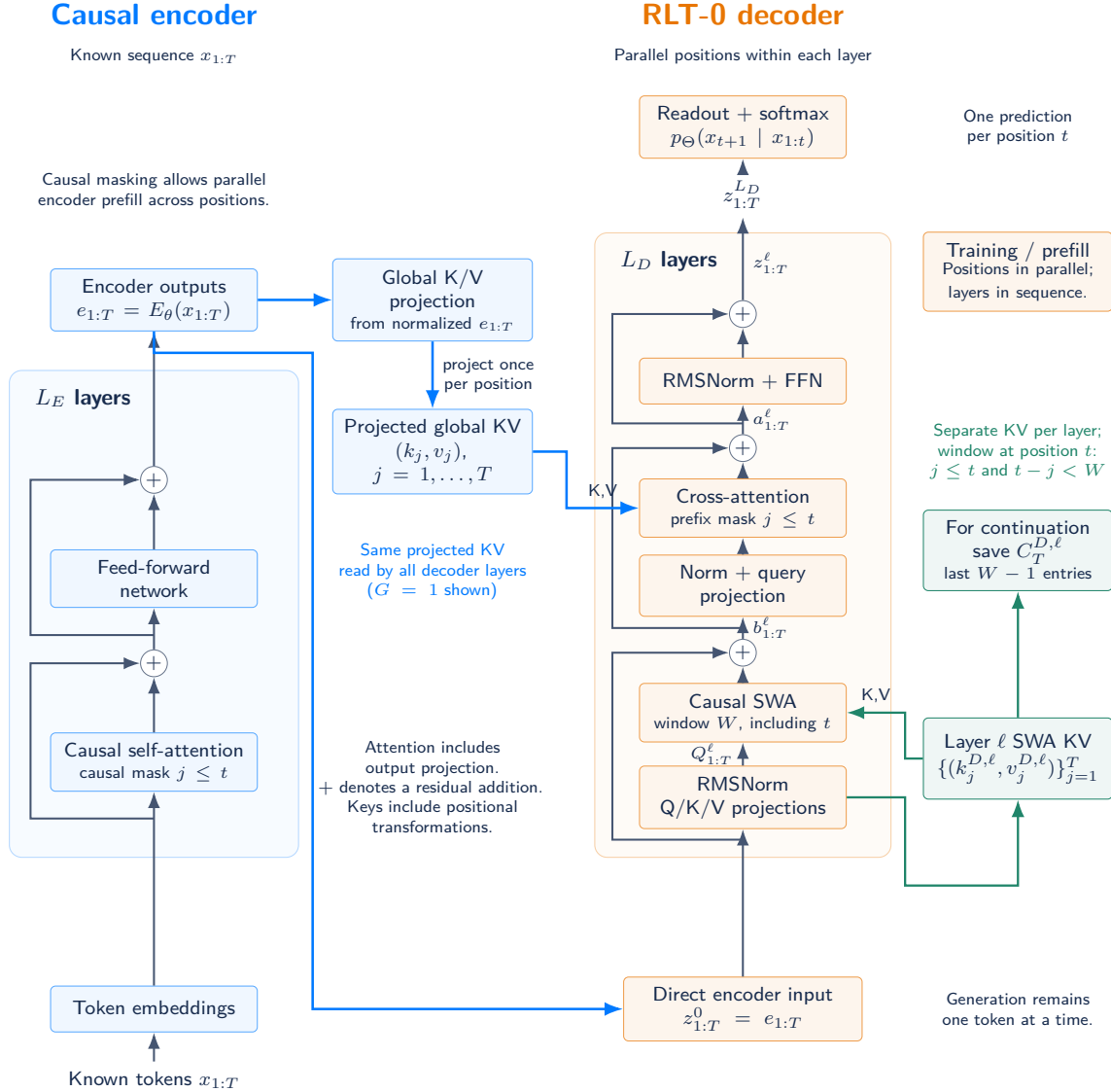


图 12 训练或提示预填充阶段的 RLT-0。解码器直接接收编码器输出， $z_{1:T}^0 = e_{1:T}$ ，门控融合与最终隐藏状态反馈路径均被移除。各虚线框中的模块按标注深度重复。当 $G = 1$ 时，所有解码器层读取同一份已投影的全局 KV；查询位置 t 可读取编码器位置 $j \leq t$ 。各解码器层构造自己的 SWA KV，并使用包含当前位置、大小为 W 的因果窗口。已知位置在每层内一起处理，各层则按深度顺序执行。增量生成保留编码器缓存、全局记忆和各层最近的 $W - 1$ 条 SWA 缓存，逐 token 进行。

表 5 固定 S_5 验证集在第 2,000 步的最终状态与前缀 token 准确率 (%), 报告种子 42、43 和 44 的均值 $\pm$ 样本 SD。

Task	Metric	RLT-1 4+4	RLT-1 5+3	RLT-1 6+2	RLT-1 7+1	RLT-1 8+0	GPT 8
S_5 , swaps	Final state	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	99.35 $\pm$ 0.81	99.61 $\pm$ 0.68	99.09 $\pm$ 0.23
S_5 , swaps	Prefix token	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	99.96 $\pm$ 0.05	99.95 $\pm$ 0.06	99.91 $\pm$ 0.06
S_5 , standard	Final state	0.78 $\pm$ 0.68	2.47 $\pm$ 1.26	2.21 $\pm$ 1.48	2.08 $\pm$ 0.98	1.82 $\pm$ 1.19	0.52 $\pm$ 0.23
S_5 , standard	Prefix token	5.57 $\pm$ 2.11	7.82 $\pm$ 0.55	7.99 $\pm$ 0.29	7.67 $\pm$ 0.54	6.47 $\pm$ 0.65	5.40 $\pm$ 1.21

J.4 完整训练快照的复现

形式任务的训练、验证与测试数据种子分别为 20260914、20260915 和 20260916。加法使用数据种子 42。每个任务的三个初始化种子共享相同的索引训练数据流和留出样本。全部 108 次运行使用基于上游提交 d1a4516 的实现，均完成 2,000 步。原生训练历史于 2026 年 9 月 17 日冻结，清单记录精确采集时间和来源哈希。

目录 `data/experiments-depth8/` 中的 `manifest.json` 和 `configs.json` 记录运行标识、设置以及来源与语料指纹。数据集包含 2,160 条验证记录和 216,000 条未经平滑的训练步骤记录。目录 `scripts/` 中的 `plot_depth8.py` 从冻结记录重新生成训练图表。固定长度 token 准确率导出包含 90 次完整形式任务运行的 1,800 条验证记录。同一脚本目录中的 `plot_token_accuracy.py` 重新生成对应图形。

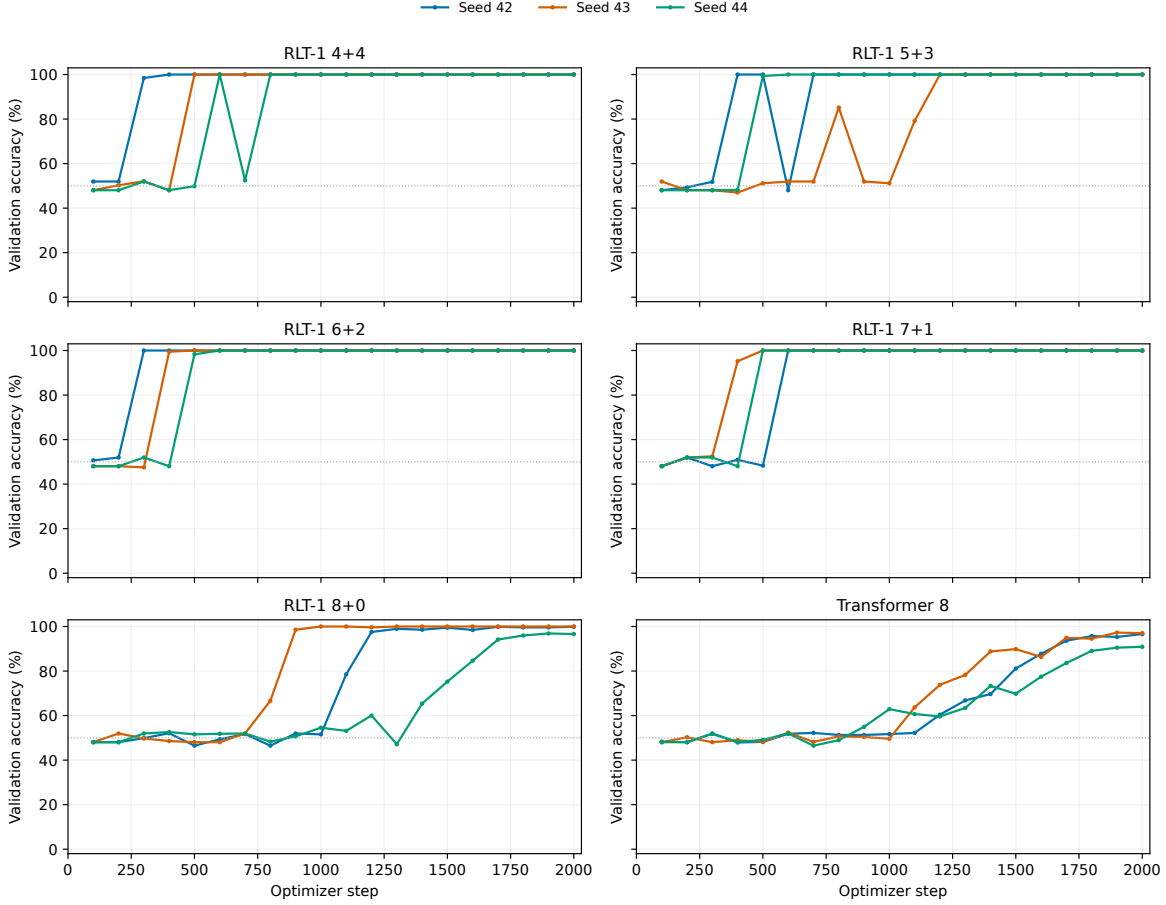


图 13 各初始化种子的奇偶校验准确率，均延伸至第 2,000 步。所有运行共享索引训练数据流与验证集，曲线保留暂时回退和学习速度差异。

J.5 长度泛化协议与补充指标

该实验使用全部 108 次完整运行，每种架构与任务均包含种子 42、43 和 44。每次运行按原生分布内验证样本损失最小选择检查点；形式任务先合并配置的验证长度，损失相同时选择最早步骤。表 6 列出所选步骤，检查点与训练结果的哈希随数据保存。测试结果不参与检查点选择。

加法每个操作数的测试位数为 9、10、11、12、14、16 和 32。奇偶校验与带括号模五运算测试长度为 32、40、48、64、96、128、192 和 256；两种 S_5 任务使用 32、48、64、96、128、192、256 和 512。无括号表达式由操作数与二元运算符交替构成，因此每个请求的偶数长度 L 映射为实际表达式长度 $L - 1$ ，即 31、39、47、63、95、127、191 和 255。形式任务长度不包含 BOS 和末尾等号；无括号模五运算加入这两个标记后有 $L + 1$ 个输入 token。共有 47 个共享测试集和 846 个模型、种子与长度组合的评估。

加法在每个位数下使用 256 对互不重复的无序操作数，按种子 20260916 从原生测试划分采样。两个操作数均恰好具有指定的位数且无前导零，逆序数字序列化与训练一致。评估计算教师强制答案 token 准确率，不执行自回归生成。每个形式任务测试集由原任务生成器和测试种子 20260916 产生，包含 1,024 条序列。所有模型与初始化在同一任务及长度下使用相同样本。共享留出集沿用上一轮基准，新生成的形式任务集合排除重复样本和已有验证、测试样本。

检查所选检查点与留出集指纹后，我们复用 697 个模型、种子与长度组合的结果，并评估剩

余 149 个组合。所有指标均从保存的逐样本预测或教师强制 token 计数重新计算。评估使用 CPU FP32、四个算子内线程和训练时记录的数值运行环境，检查点权重保持固定。新评估的批量为 32，复用的原生形式任务评估批量为四。此前批量检查对每种 S_5 任务使用 32 条长度 512 的序列，RLT-1 4 + 4 在两种批量下的全部 32,768 个状态预测一致。仅做教师强制的加法评估器也在 32 对共享的 32 位操作数上，复现了 RLT-1 4 + 4 与 Transformer 8 保存的逐样本教师强制 token 计数。

对每个测试长度，先计算各初始化的分数，再将上述均值与 SD 公式中的训练步骤替换为长度。误差棒表示共享测试样本上三个初始化之间的样本 SD。图 14 补充报告六任务 token 准确率；三种 S_5 评分规则的比较见第 3.5 节（图 5）。

目录 `data/experiments-depth8-generalization/` 包含 846 条逐种子记录、282 个聚合点、检查点与数据集指纹，以及验证报告。可移植脚本 `scripts/plot_generalization.py` 检查种子和长度覆盖、计数推导的分数、样本 SD，以及检查点与训练快照的对应关系，再生成图表。

Length generalization · 8 layers · seeds 42, 43, 44

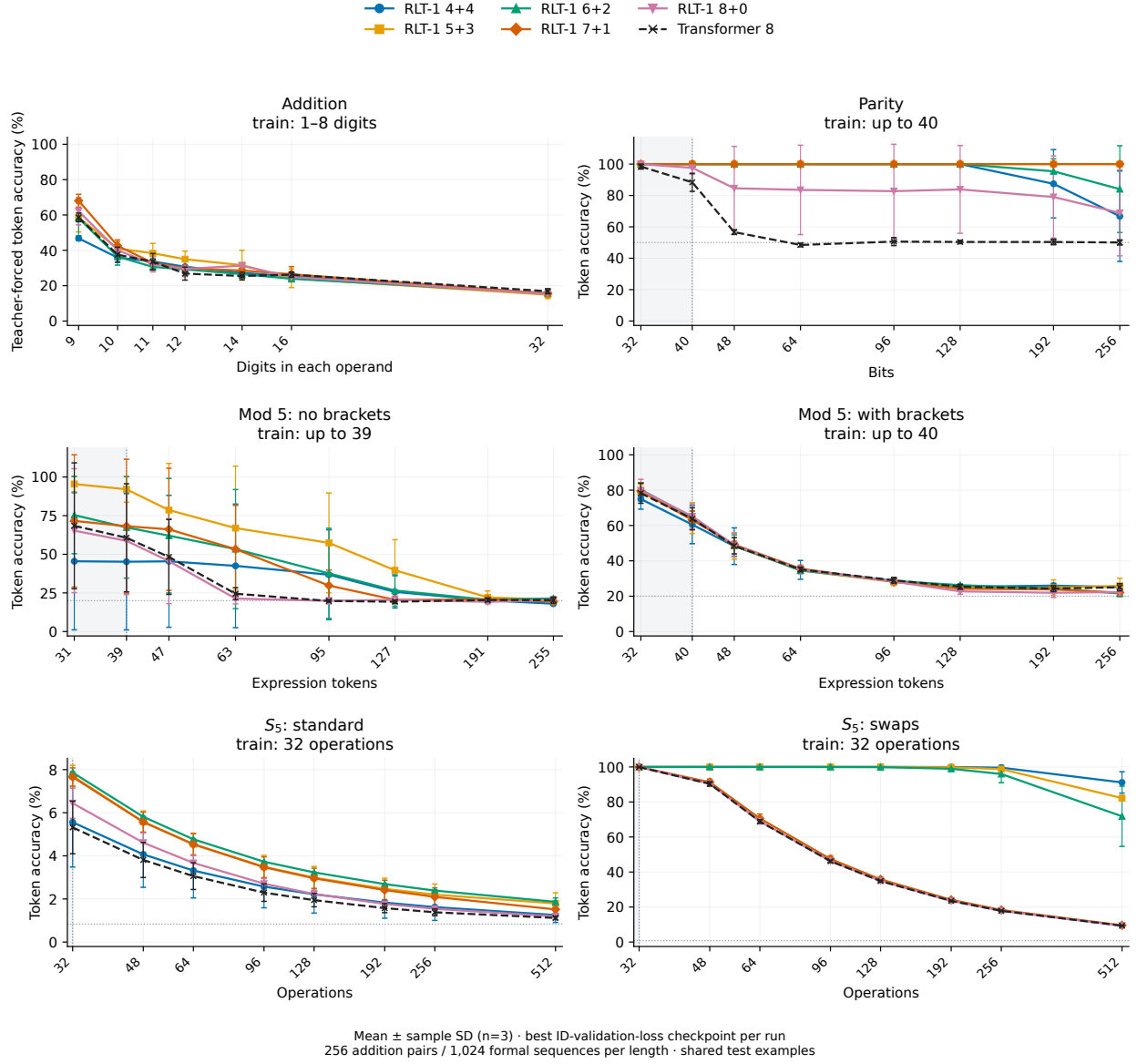


图 14 不同测试长度上的 token 准确率，三个初始化种子。检查点与共享样本同图 4。加法对答案 token 使用教师强制， S_5 对每个前缀状态评分，奇偶校验与模五运算各对一个最终标签评分。点和误差棒表示种子 42、43 和 44 的均值 $\pm$ 样本 SD，灰色区域表示训练长度。

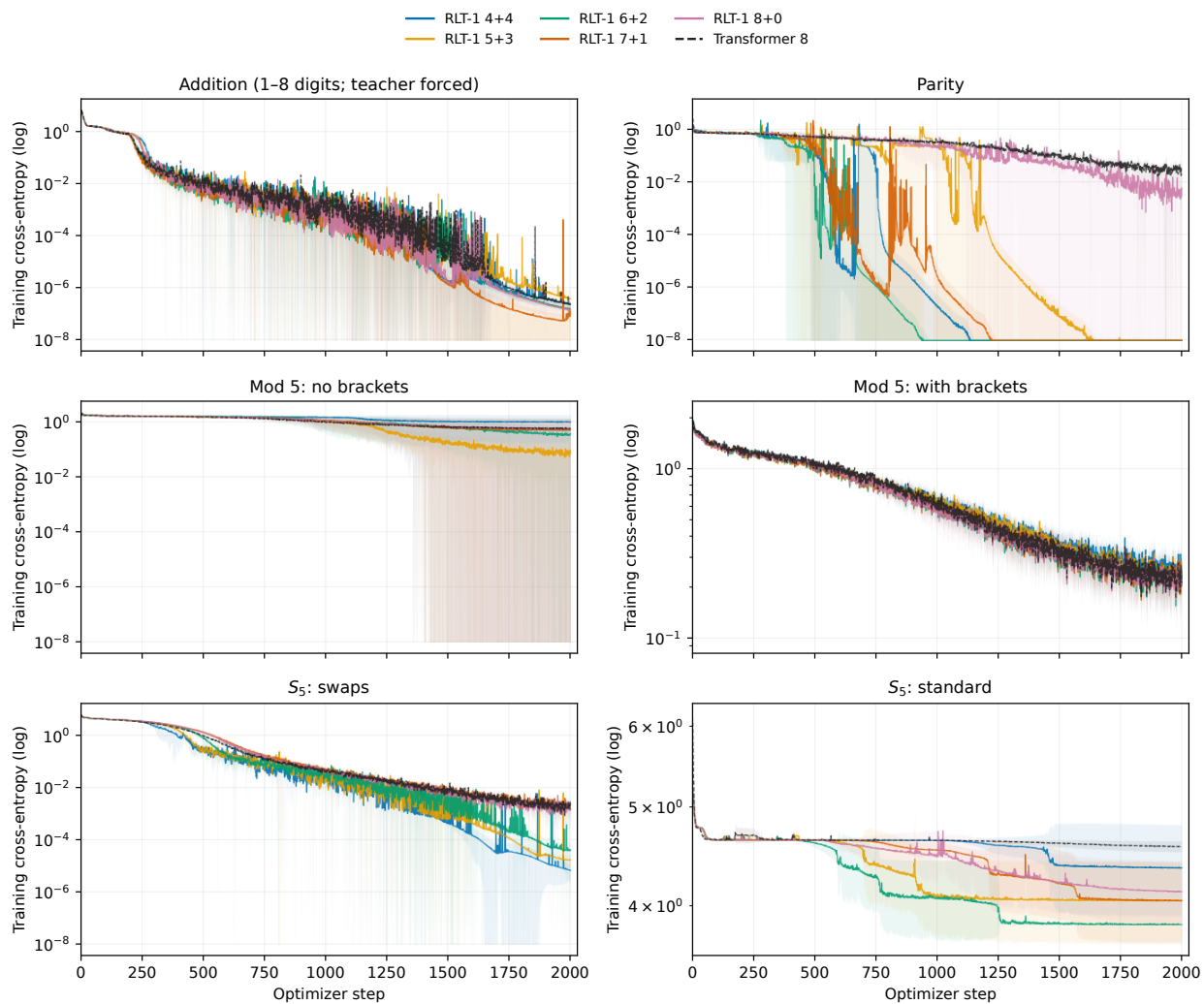


图 15 未经平滑的全局批量训练交叉熵，每个任务均展示三个种子的均值 $\pm$ 样本 SD。在对数轴上，低于 10^{-8} 的均值或阴影边界按 10^{-8} 显示。损失在优化器更新前计算，记录的梯度范数为裁剪前的值。各任务的监督方式不同，应在同一任务内比较损失。

表 6 按最低分布内验证损失选取的长度泛化检查点步骤。每次来源运行均完成 2,000 步，分别列出种子 42、43 和 44。

Task	Seed	RLT-1 4+4	RLT-1 5+3	RLT-1 6+2	RLT-1 7+1	RLT-1 8+0	GPT 8
Addition	42	2000	2000	2000	2000	2000	2000
	43	2000	2000	2000	2000	2000	2000
	44	2000	2000	2000	1900	2000	2000
Parity	42	900	1600	800	1500	2000	2000
	43	1100	2000	900	1500	1900	1900
	44	1400	1000	1200	1100	2000	2000
Mod 5, no brackets	42	800	2000	2000	2000	2000	400
	43	800	2000	2000	800	400	2000
	44	2000	2000	2000	2000	2000	1900
Mod 5, brackets	42	2000	2000	2000	2000	2000	1900
	43	2000	2000	2000	2000	2000	2000
	44	2000	2000	2000	2000	2000	2000
S_5 , standard	42	1800	2000	2000	2000	2000	2000
	43	2000	2000	2000	2000	2000	2000
	44	2000	2000	1800	2000	2000	2000
S_5 , swaps	42	2000	2000	2000	1900	2000	2000
	43	2000	2000	2000	2000	2000	1900
	44	2000	2000	2000	1900	2000	2000

J.6 长度为 32 和 33 时的 token 准确率曲线

图 16 比较奇偶校验、带括号模五运算和两种 S_5 任务在长度 32 时的 token 准确率，以及无括号模五运算在长度 33 时的 token 准确率。无括号表达式的数字与二元运算符交替出现，因此有效内容长度为奇数；33 是已配置验证长度中最接近 32 的值。完整快照包含 90 次运行的 1,800 个验证检查点：六种架构、五个形式任务、三个种子，每次均完成 2,000 步训练。长度为 32 时， S_5 统计 256 条序列中的 8,192 个前缀状态；奇偶校验与模五运算每条序列评分一个最终标签。

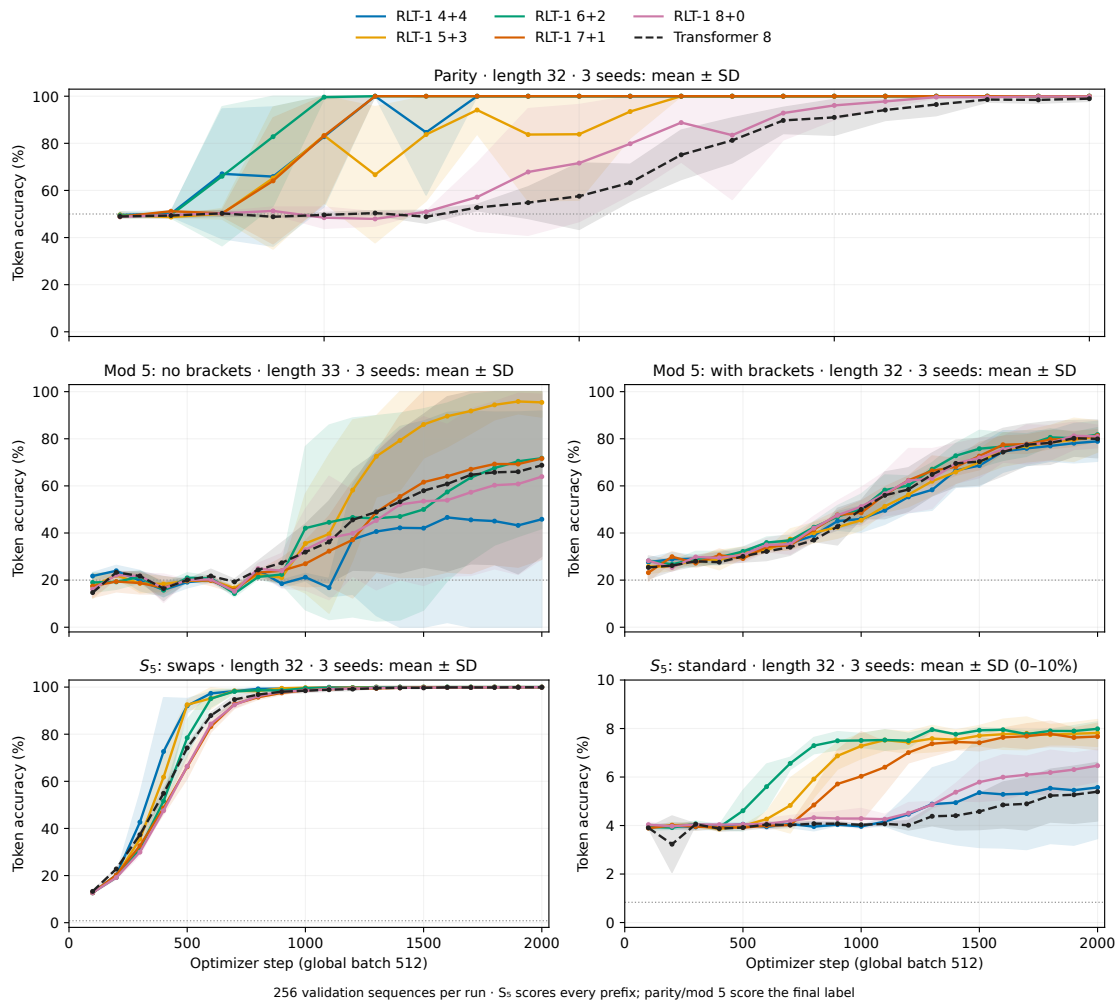


图 16 五个形式任务的验证 token 准确率，每个任务三个种子。无括号模五运算的有效内容长度为 33，其余面板为 32。每个点使用同一优化器步骤的全部三个初始化种子；阴影为均值 ± 样本 SD，并裁剪到准确率范围内。 S_5 对所有前缀状态评分，奇偶校验与模五运算对最终标签评分。所有曲线均延伸至第 2,000 步。标准 S_5 使用较窄的纵轴范围，横向点线表示均匀预测准确率。

J.7 十六层模型的加法泛化

我们使用初始化种子 42，单独评估 RLT-1 $8+8$ 、 $9+7$ 、 $10+6$ 、 $11+5$ 、 $12+4$ 、 $13+3$ 、 $14+2$ 、 $15+1$ 和 $16+0$ ，以及十六层 Transformer。模型使用与八层加法运行相同的混合 1–8 位训练语料、宽度 512、FFN 宽度 1,365、四个注意力头、优化器设置、全局批量 512 和微批量 32；RLT-1 使用 TBPTT 128。十六层 RLT-1 配置的参数量为 51.27–56.00M，Transformer 为 50.49M。在附录 J.5 的七个位数下，十个模型使用完全相同的 256 对操作数，新增 70 个模型与长度组合的评估。评估使用 CPU FP32，同时报告教师强制 token 准确率和贪心解码的完整答案准确率；后者要求答案及其后的 EOS 均正确。逐点 95% Wilson 区间描述测试样本对之间的完整答案不确定性。

检查点从 2026 年 9 月 16 日 15:32:01 UTC 快照时已有的检查点中选择，以最低分布内验证样本损失为准，损失相同时选择最早步骤。四次运行尚未完成： $8+8$ 、 $9+7$ 、 $10+6$ 和 $11+5$ 分别达到第 1,200、1,400、1,500 和 1,800 步；所选检查点分别位于第 1,200、1,300、1,400 和 1,800 步。其余六次运行均已完成 2,000 步，并选中最终检查点。十个所选检查点在原生混合 1–8 位验证集上的完整答案准确率均为 100%。图中用剑号标记未完成的运行，并在图例中注明所选检查点步骤。

图 17 表明，在九至 32 位的每个测试位数下，所有 RLT-1 配置在 256 对操作数中均未生成完全正确的答案。十六层 Transformer 在九位时生成三个完全正确的答案 (1.17%)，在更长的测试位数下均为零。教师强制答案 token 准确率较高：九位时 Transformer 为 84.51%，各 RLT-1 配置为 47.32–65.82%。32 位时，十个模型的该项指标均处于 13.95–16.21%。token 与完整答案准确率之间的差距反映了在训练范围之外生成整段正确答案的困难。

图 18 比较 $8+8$ 与 $4+4$ ，并纳入各深度的 Transformer 基线。RLT-1 $8+8$ 在九位时的教师强制 token 准确率为 51.48%， $4+4$ 为 47.17%；但在十至 32 位的每个测试位数下， $8+8$ 都更低。训练预算并不相同： $8+8$ 的检查点位于第 1,200 步， $4+4$ 和两个 Transformer 则位于第 2,000 步。

固定数据保存在 `data/arithmetic-generalization-depth16/`，包含全部 70 项新测量、42 项重新验证的八层加法测量、检查点身份、共享留出集哈希，以及预测级验证报告。脚本 `scripts/plot_arithmetic_depth16.py` 可重新生成这两张独立图形。

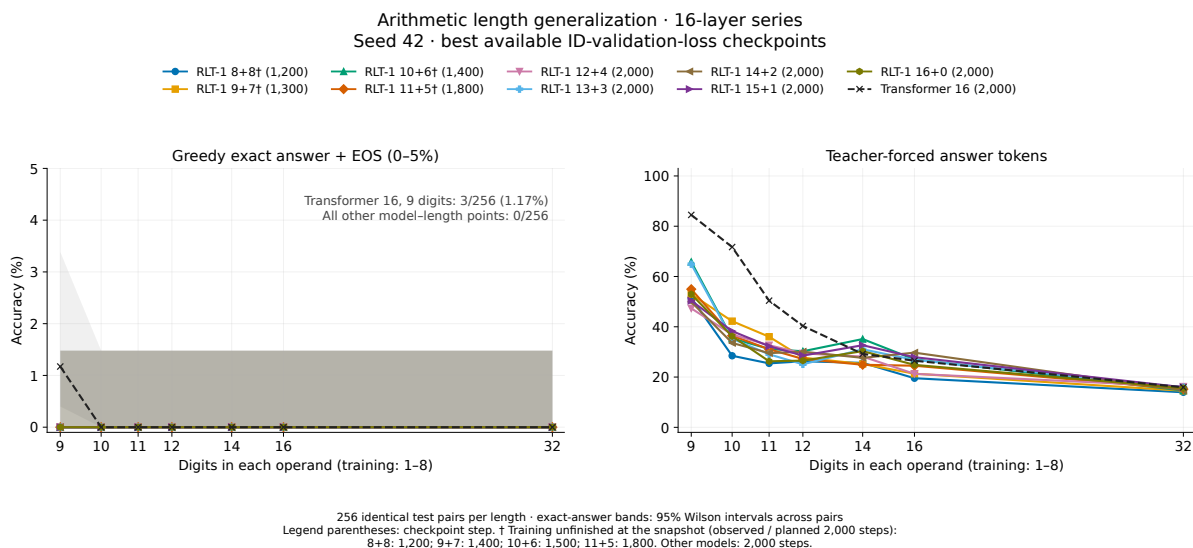


图 17 十六层系列的加法泛化，种子 42。九种 RLT-1 深度分配和十六层 Transformer 使用八层实验中的七个留出集，每个位数包含 256 对操作数。左面板使用 0-5% 的纵轴范围，报告严格的贪心答案加 EOS 准确率；右面板使用 0-100% 的范围，报告教师强制答案 token 准确率。完整答案的阴影带表示跨测试样本的逐点 95% Wilson 区间。图例括号给出当前最佳分布内验证损失检查点的步骤；剑号标记快照时尚未完成 2,000 步训练的四次运行。

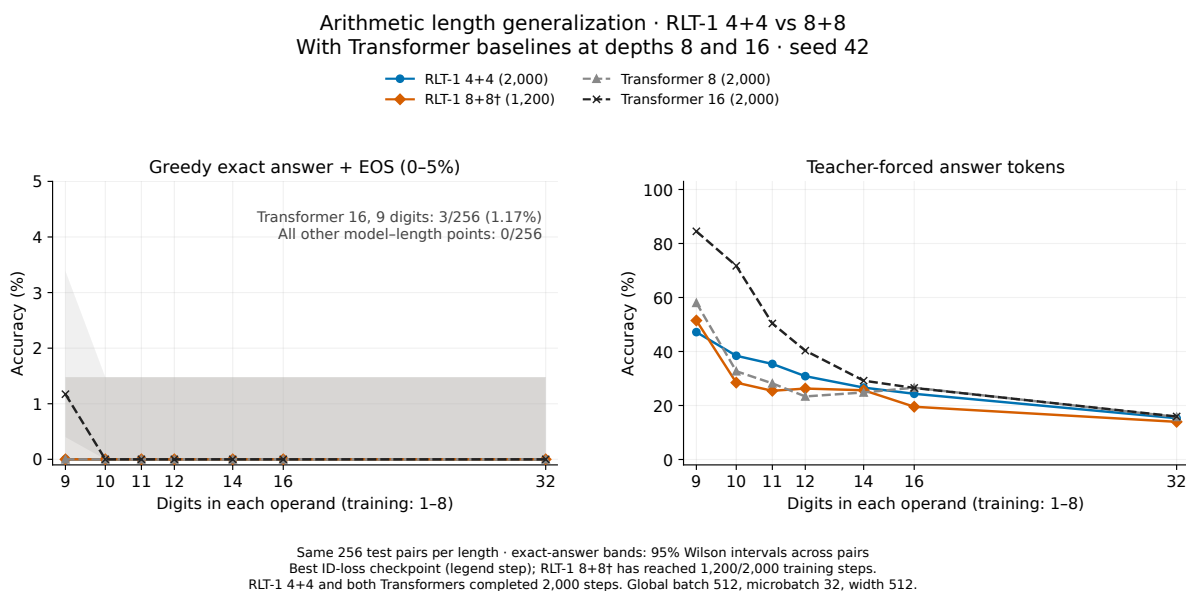


图 18 RLT-1 4+4 与 8+8 的加法泛化，以及两个深度的 Transformer 基线。四个种子 42 模型使用与图 17 相同的测试样本和评分规则。RLT-1 8+8 使用截至第 1,200 步的最佳检查点，其余三个模型已完成 2,000 步并选择该步骤的检查点。训练预算因此不同；Wilson 阴影带描述跨测试样本的不确定性，不描述训练初始化间的差异。